Republic of Tunisia
Ministry of High Education and Scientific Research
***
University of Monastir
High Institute of Computer Science and Mathematics of Monastir
Department of Technology
***

Graduation Project

In order to obtain the
National Diploma of Engineering in Applied Sciences and Technology

Major:
Electronics: Microelectronics

Presented by:

Najd ELAOUD and Selim TRIKI

Design and development of the IOBOX-BWS industrial platform for data acquisition, processing, and control

---

Presented 11 **July** 2026 **in front of the jury members :**

Mr. Nejib HASSEN	President
Mr. Khaled ISSA	Lecturer
Mr. Abdessalem BEN ABDELALI	Academic Supervisor
Mr. Jamel HMIDI	Industrial Supervisor

Academic year: 2025/2026

ACKNOWLEDGEMENTS

I am humbled and grateful to all who have helped me translate these ideas into something concrete that goes far beyond a simple level.

I would like to express my sincere gratitude to my academic supervisor, **Mr. Abdessalem BEN ABDELALI**, for his guidance throughout this project, for reviewing my work, and for his valuable remarks and suggestions that helped improve the quality of this report.

I would also like to express my deepest appreciation to my industrial supervisor, **Mr. Jamel HMIDI**, for his continuous support, technical guidance, and availability throughout the internship. His expertise, advice, and encouragement were invaluable to the successful completion of this project.

Last but not least, I am deeply grateful to all the jury members **Mr. Nejib HASSEN** and **Mr. Khaled ISSA** for agreeing to evaluate this work.

Without your support, this accomplishment would not have been achieved. I really appreciate it.

DEDICATION

To my precious family, I would never be where I am now without you.

To my dear father ***Noureddine****:*
Thank you for your hard work, guidance, and the values you have instilled in me throughout my life. Your sacrifices and dedication have always inspired me to strive for success. May God bless you with health, happiness, and a long life.

To my dear mother ***Soufia****:*
You have done more than a mother can do to ensure that her children follow the right path in their lives and studies. I dedicate this work to you as a testimony of my deep love. May God, the Almighty, preserve you and grant you health, long life, and happiness.

To my dear sister ***Nibras****:*
Thank you for your constant support, encouragement, and affection. Having you by my side has always been a source of strength and motivation.

To my dear Friends, Words can't express how grateful I am to have your friendship in my life. Thank you for supporting me, accepting me, and being wonderful friends.

Lastly, to all the kind people I met during my six years here, to all the institute students, professors, administrators and workers, thank you for being part of my exciting university career.

Najd ELAOUD

DEDICATION

This work is dedicated to the people whose love, support, and encouragement have shaped my journey and made this achievement possible.

To my dear father ***Lotfi****:*
This project and my success in my academic journey could not have been achieved without your presence in my life. I am deeply grateful for the comfort and opportunities you provided for us, and for teaching me that with dedication and perseverance nothing is impossible. I hope that one day I can repay you, even in a small way, for all that you have done.

To my dear mother ***Monia****:*
The love you gave us made us stronger, and the dedication and support you offered whenever we fell down or felt low is the most precious gift in life. Thank you for being the light that guided us and for showing us the strength of unconditional love.

To my dear sister ***Ahlem****:*
Thank you for your emotional support and I am truly grateful for the strength and companionship you have given me.

To my dear brother ***Youssef****:*
I cannot wait to see you thrive and shine in life, overcoming every turbulence along the way. I will always be here as your mentor whenever you need guidance.

To my dear ***friends*** *,Thank you for all the support you have given me and for the joy you bring into my life. I am truly grateful that you accept every side of my personality, and I cherish the real human connection we share. Seeing you succeed in life is part of my own dreams, and I look forward to celebrating your achievements alongside you.*

Selim TRIKI

ABSTRACT

The development of flexible and interoperable industrial platforms is essential for addressing the growing integration challenges of Industrial Internet of Things (IIoT) systems. This report presents the design and development of the IOBOX, a configurable industrial Input/Output platform intended to simplify the acquisition, processing, control, and exchange of data between sensors, actuators, and supervisory systems.

The developed platform is based on an STM32 microcontroller and integrates multiple industrial communication interfaces, peripheral drivers, and sensor management modules within a modular software architecture. The implementation emphasizes hardware abstraction, software reusability, and scalability, enabling efficient integration of heterogeneous devices while reducing development effort.

The resulting solution provides a versatile and reusable platform for industrial monitoring, automation, and IoT applications. By improving interoperability and simplifying system integration, the IOBOX contributes to the development of more efficient and scalable industrial solutions.

Keywords: Industrial IoT, Embedded Systems, STM32, IO-Box, Data Acquisition, Industrial Communication Protocols, Automation, Software Architecture.

RESUME

Le développement de plateformes industrielles flexibles et interopérables est devenu essentiel pour répondre aux défis croissants d'intégration des systèmes de l'Internet Industriel des Objets (IIoT). Ce rapport présente la conception et le développement de l'IOBOX, une plateforme industrielle d'Entrées/Sorties configurable destinée à simplifier l'acquisition, le traitement, le contrôle et l'échange de données entre les capteurs, les actionneurs et les systèmes de supervision.

La plateforme développée repose sur un microcontrôleur STM32 et intègre plusieurs interfaces de communication industrielle, pilotes de périphériques et modules de gestion de capteurs au sein d'une architecture logicielle modulaire. L'implémentation met l'accent sur l'abstraction matérielle, la réutilisabilité logicielle et l'évolutivité, permettant une intégration efficace de dispositifs hétérogènes tout en réduisant les efforts de développement.

La solution obtenue constitue une plateforme polyvalente et réutilisable pour les applications de supervision industrielle, d'automatisation et d'IoT. En améliorant l'interopérabilité et en simplifiant l'intégration des systèmes, l'IOBOX contribue au développement de solutions industrielles plus efficaces et évolutives.

Mots-clés : Internet Industriel des Objets (IIoT), Systèmes Embarqués, STM32, IOBOX, Acquisition de Données, Protocoles de Communication Industrielle, Automatisation.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

GLOSSARY OF ACRONYMS

BSP: Board Support Package

UART : Universal Asynchronous Receiver-Transmitter

LIN: Local Interconnect Network

USB : Universal Serial Bus

CAN : Controller Area Network

I2C : Inter-Integrated Circuit

SPI : Serial Peripheral Interface

DAC : Digital to Analog Converter

ADC : Analog to Digital Converter

PWM : Pulse Width Modulation

DMA : Direct Memory Access

GPIO : General Purpose Input Output

PID : Proportional Integral Derivative

GENERAL INTRODUCTION

Industrial systems are increasingly relying on connected devices capable of acquiring, processing, and exchanging information in real time. In such environments, sensors, actuators, and supervisory systems must interact efficiently through various communication interfaces and protocols. However, integrating these heterogeneous components often requires significant development effort, particularly when flexibility, scalability, and interoperability are required.

To address these challenges, Be Wireless Solutions (BWS) launched the development of the IOBox platform, a configurable industrial Input/Output system designed to serve as a universal interface between field devices and monitoring or control systems. The platform is intended to simplify the integration of sensors and actuators while providing data acquisition, signal processing, control, and communication functionalities within a single solution.

The IOBox supports multiple industrial communication protocols and offers both analog and digital input/output capabilities. Its modular architecture enables adaptation to various industrial applications, ranging from environmental monitoring and energy management to process automation and equipment supervision. By centralizing these functionalities in a single platform, the IOBox reduces deployment complexity and accelerates the development of industrial IoT solutions.

The objective of this end-of-study project is to participate in the design and implementation of the IOBox platform. The work focuses on the development of embedded software components, communication interfaces, peripheral drivers, sensor integration, and system architecture required to ensure reliable operation in industrial environments.

This report, which crowns the work entrusted to us, is structured around three chapters:

Chapter 1 presents the general context of the project, the host company, the problem statement, the project objectives, and the adopted development methodology.

Chapter 2 introduces the general concept of the IOBox platform and describes the hardware components, software tools, communication protocols, sensors, and peripherals used throughout the project.

Chapter 3 details the design and implementation of the platform, including the software architecture, peripheral configuration, driver development, communication management, sensor integration, and system validation.

CHAPTER 1

GENERAL PROJECT CONTEXT

1.1 Introduction

This chapter gives an overview of the professional and academic context of our graduation project. First, it starts by introducing the host company, Be Wireless Solutions (BWS), covering its structure, its line of work, and the main IoT solutions it offers. Then we'll outline the project's requirements, starting with explaining the problem that was identified, then explaining how the IOBOX platform came out of it, the development method used, and the project's goals and the effects of the project.

1.2 Academic Institution

The Higher Institute of Computer Science and Mathematics of the University of Monastir (ISIMM) was created by decree number 1623 on July 9, 2002. It is a public scientific establishment of higher education under the authority of the Ministry of Higher Education and Scientific Research [1].

ISIMM offers engineering courses in Computer Science, Mathematics and Electronics. This graduation project was developed within the Electronics department, specialized in Microelectronics, following the requirements for the National Diploma in Engineering in Applied Sciences and Technology.

1.3 Host Company Presentation

1.3.1 Company Overview

Be Wireless Solutions (BWS) is a technology group founded in 2016, specializing in the development of IoT/AI solutions to optimize the use of resources such as electricity, water, and fuel. Operating across three continents, BWS supports industrial companies, logistics and transportation companies, utilities, and local governments in their transition towards more sustainable and efficient resource management.[2]

Figure 1.1: Be Wireless Solutions (BWS) logo

1.3.2 Organization Chart

BWS has different departments that work together during the whole process of creating industrial and IoT solutions. These departments are management, research and development, hardware design, embedded software development, cloud services, technical support, and business development.

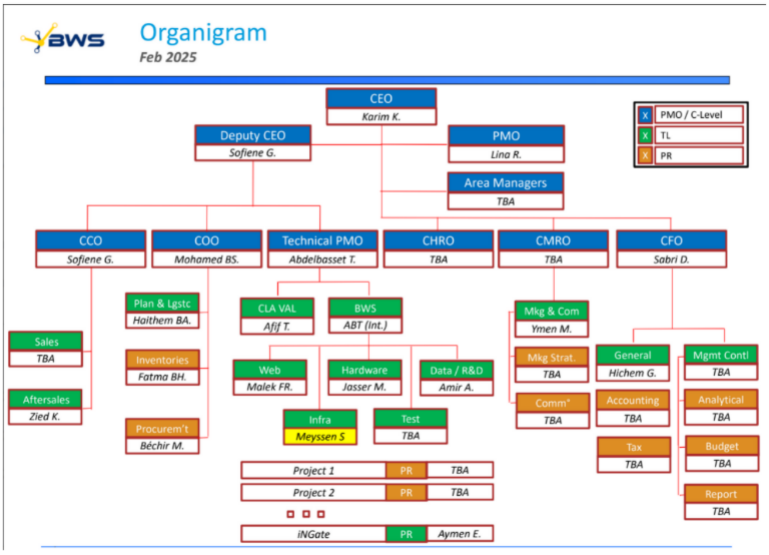

Figure 1.2: Organization chart of Be Wireless Solutions

1.3.3 Main Activities and Services

BWS creates many different industrial IoT solutions that help make operations more efficient, manage resources better, and keep track of processes more effectively.

Energy Monitoring and Management

BWS provides intelligent energy monitoring systems capable of collecting, analyzing, and visualizing electricity consumption data in real time. These solutions help organizations identify energy-intensive equipment, optimize consumption, reduce operational costs, and improve environmental performance.

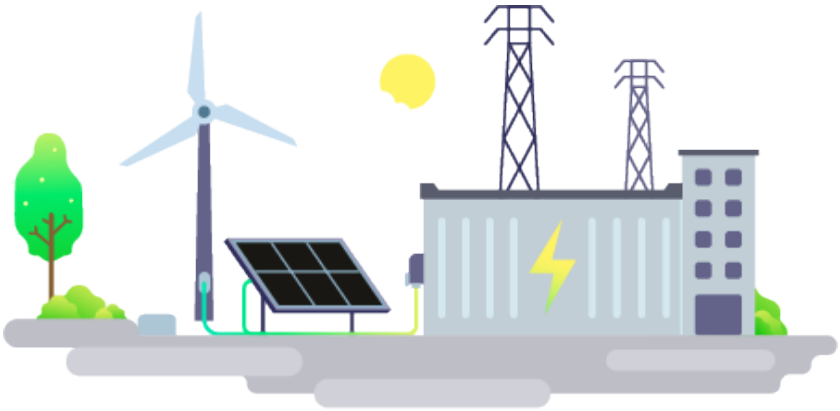

Figure 1.3: Electricity consumption monitoring solution

Water Monitoring Solutions

The company provides smart water management solutions that allow for remote reading of meters, detecting leaks, monitoring unusual water use, and helping to use water resources more efficiently.

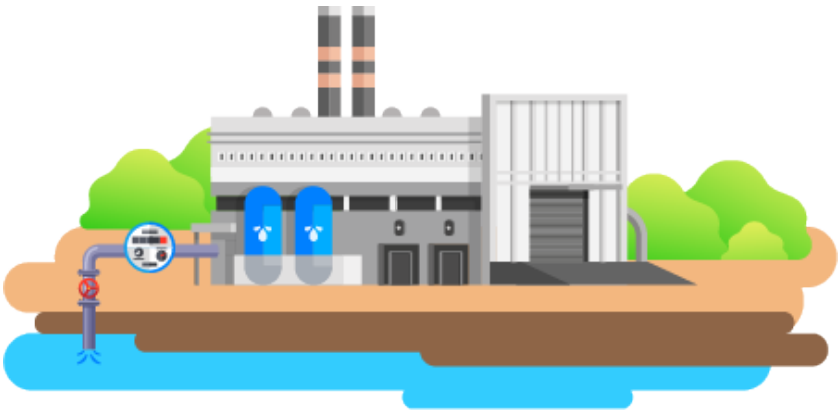

Figure 1.4: Water consumption monitoring solution

Fleet and Fuel Management

BWS develops fleet management systems that provide real-time vehicle tracking, fuel consumption analysis, route optimization, and driver behavior monitoring.

Figure 1.5: Fleet and fuel management solution

Remote Monitoring and Equipment Control

The company provides remote supervision solutions that allow operators to monitor industrial equipment, detect anomalies, generate alerts, and perform remote control operations through dedicated software platforms.

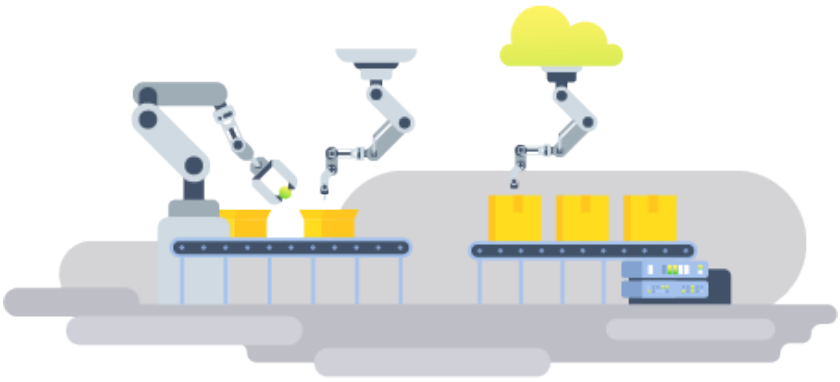

Figure 1.6: Remote equipment monitoring and control

Cold Chain Monitoring

BWS also offers cold-chain monitoring systems intended for warehouses, refrigerated transportation, and pharmaceutical storage facilities. These solutions ensure

compliance with temperature requirements and provide instant alerts when abnormal conditions occur.

Figure 1.7: Cold-chain monitoring solution

1.4 Project Specification

1.4.1 Problem Statement

Industrial IoT systems are usually composed of a multitude of sensors, actuators, and communication interfaces. For each client project BWS developed dedicated firmware for each new setup; every time there was a new set of peripherals, they would need to be reconfigured and the drivers rebuilt. This approach resulted in slower development, limited reuse of code between projects, and made system maintenance harder when hardware changed.

To address these issues, BWS initiated the IOBOX project. The aim was to develop a flexible, hardware-independent platform that can support different setups on one common firmware base. In industrial environments there are often different communication protocols and interface standards. Handling these through custom solutions adds complexity and makes it harder to scale the system.

The goal is to provide a customizable and reusable system for industrial data collection and control. This system aims to make the addition of peripherals easier while keeping the system flexible, easy to relocate between setups, and easier to maintain.

1.4.2 Presentation of the IOBOX Platform

The IOBOX is an industrial flexible system that helps to interconnect sensors, actuators and communication tools in industrial environments.

It adopts a modular approach that distinguishes the parts dependent on specific hardware from the parts that manage the main functions. This is achieved through specific software tools such as drivers, a board support package, and Manager, which facilitate use in different systems and maintain an easy update.

The IOBOX can interface with various types of industrial equipment such as analog and digital inputs and outputs, communication ports and measurement devices. Its software configuration can be adapted to various industrial requirements without major changes in the firmware.

By connecting the hardware and the software in a straightforward way, the IOBOX simplifies the assembly and the use in industrial environments.

1.4.3 Project Objectives

The key objective of this project is to develop the IOBOX platform which will collect, handle, control, and transmit data in real-time.

To be more specific, the project aims to:

- Developing a scalable framework that is capable of acquiring the data and controlling it according to industry needs.
- Provide a unified software abstraction layer for heterogeneous peripherals.
- Delivering programmable analogue and digital input-output features.
- Supporting industrial communication protocols such as Modbus RTU via RS485, CAN, LIN, and multi-channel UART, ensuring the compatibility with other industrial devices.
- Develop a modular software architecture based on abstraction layers and reusable software components.
- Design the software architecture to allow new sensors, actuators, and communication interfaces to be added with minimal modification to the existing code.

The resulting platform will lower development costs and reduce time-to-market by offering a reusable embedded architecture that adapts to various industrial settings.

1.4.4 Development Methodology

The project development was based on the V-Model approach, a systematic development framework widely used in embedded and industrial applications in order to preserve traceability throughout the system's lifecycle pairing each development stage

with a matching validation phase.

The project activities were organized according to the different phases of the V-Model in which the requirements-gathering phase focused on identifying the platform capabilities and technical constraints required by BWS. These requirements formed the basis for the system architecture that was designed, including the definition of the four-layer software structure and the selection of the hardware components. The detailed design stage then focused on the BSP modules, communication manager and sensor drivers.

After the design work was completed, the implementation phase involved coding BSP drivers, sensor management systems, communication protocols, and application-level functionalities. The project's validation included testing each BSP module individually, verifying sensor data acquisition and Modbus communication in an integrated environment, and performing complete system-wide testing to confirm that all operational requirements were met.

Table 1.1: Application of the V-Model to the Project

V-Model Phase	**Project Activity**
Requirements Analysis	Platform specifications derived from BWS requirements.
System Design	Hardware selection and software architecture definition.
Software Design	BSP API design and manager module interface definition.
Implementation	Development of BSP drivers, sensor managers, Modbus communication, CAN communication, and filtering algorithms.
Unit Testing	Validation of individual BSP modules and drivers.
Integration Testing	Verification of sensor chains and end-to-end Modbus communication.
System Testing	Complete platform validation on the target hardware.

The ascending branch focuses on checking and making sure everything works correctly through unit testing, integration testing, system testing, and final acceptance

testing.

The use of the V-Model helps improve the quality of the software, lowers the risks during development, and makes it easier to manage the project all through its different stages.

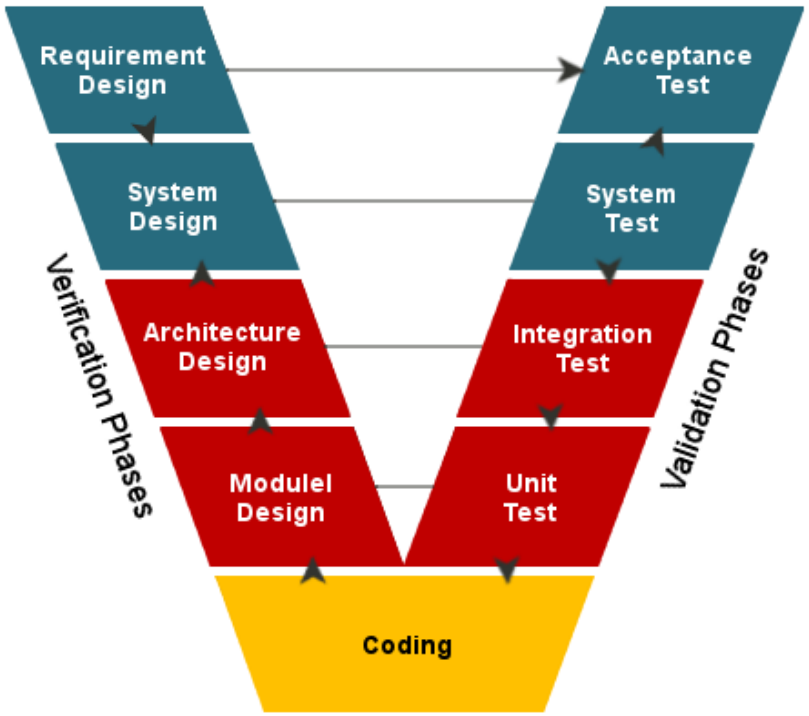

Figure 1.8: V-Model development lifecycle

1.5 Conclusion

This chapter explained the main structure of the project, starting by introducing the academic institution and the hosting company. Next, we presented the project's framework by describing the problem it aims to solve, its objectives, and the requirements that guided it's development. Finally, we outlined the methodology adopted throughout the project.

The next chapter talks about the technical setup of the IOBox platform. It covers the physical parts of the system, the software tools used for development, the ways devices communicate with each other, and the extra parts of the system that support the main functions.

CHAPTER 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

Following the detailed introduction of the project in Chapter 1, this chapter presents the technical environment and the main components used in the development of the IOBOX platform. It introduces the hardware and software resources that form the basis of the system, including the STM32G474 microcontroller, external peripherals, sensors, development tools, and communication protocols.

2.2 General Concept of the IOBOX Platform

As indicated in the previous chapter, the project's goal is to develop a configurable industrial input/output system designed to facilitate the integration of sensors, actuators, and industrial communication interfaces within IoT and automation systems.

The platform serves as an intermediary between field devices and supervisory systems by acquiring data from sensors, processing the collected information, controlling external devices and communicating using various communication protocols.

The use of a modular architecture allowed the integration of different types of hardware modules and software functionalities according to application needs. This flexibility makes the IOBOX suitable for a wide range of industrial monitoring and data acquisition applications.

2.3 IOBOX Hardware architecture

The IOBOX platform provides a set of analog, digital, and mixed-signal I/O resources. It includes four analog input channels connected to the ADC peripherals and seven digital inputs. On the output side, the device offers seven digital outputs, four PWM channels for actuator control, and DAC-based outputs capable of generating industrial 4–20 mA current signals allowing direct interaction with industrial control and monitoring systems.

The IOBOX supports multiple communication interfaces also to ensure compatibility with industrial and IoT environments, including USB for data exchange, SPI for high-speed communication and I2C buses for sensor integration and peripheral expansion. Industrial connectivity is handled through the following transceivers: the TCAN330GD for CAN FD, the TJA1020 for LIN and the THVD1410 for RS485 for a robust long-distance communication and support of Modbus RTU.

For a more flexible communication, we added the TCA9548A I^2C multiplexer that expands a single I^2C bus into eight independent channels and the PCF8574 I/O expander to drive the CD74HC4051 multiplexers for UART communication which allows us to select from multiple UART channels using the same UART bus.

Figure 2.1 shows the overall architecture of the IOBOX platform that is centered on the STM32G474MBTx microcontroller.

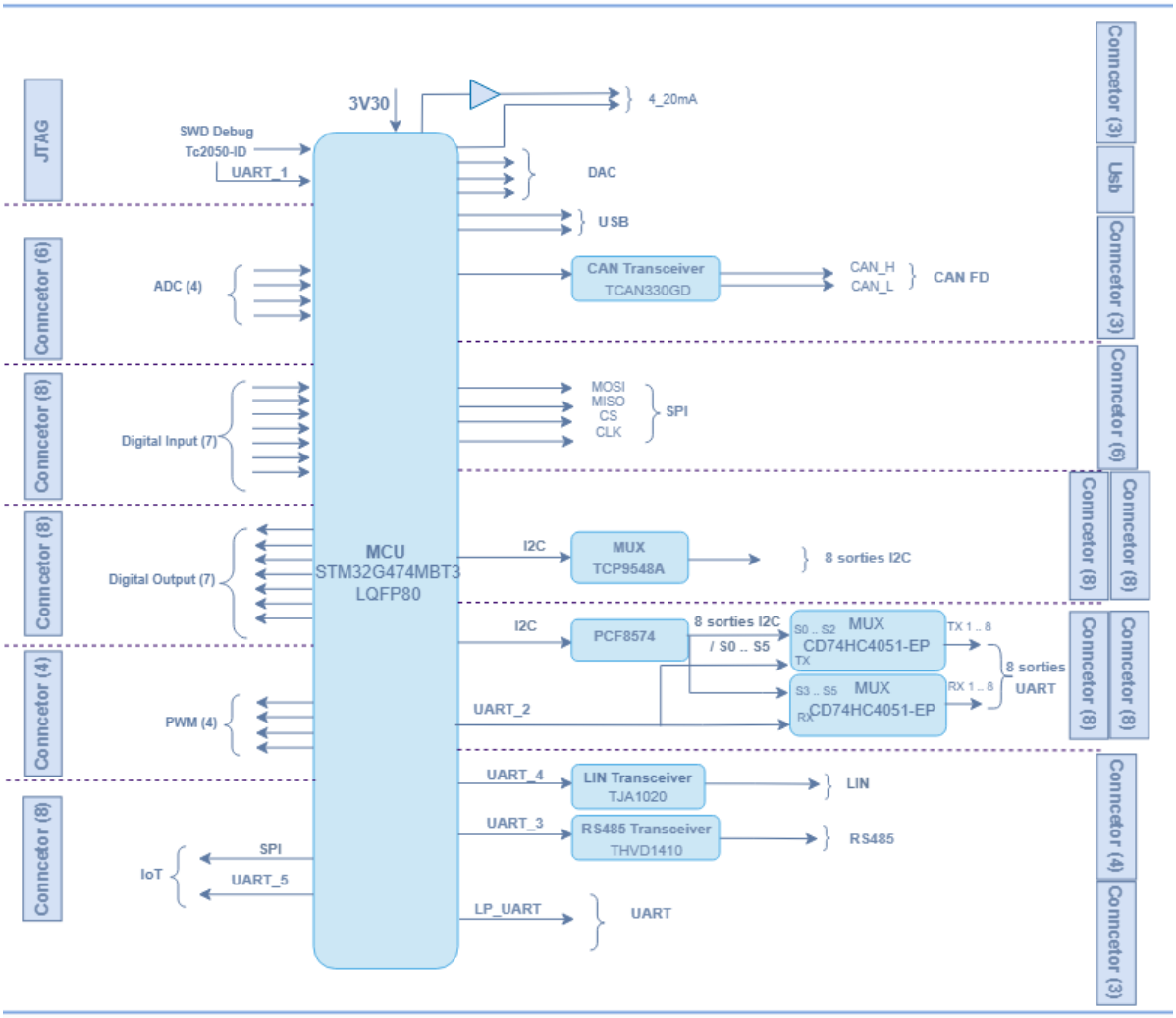

Figure 2.1: Hardware architecture of the IOBOX platform

2.4 Hardware components

2.4.1 STM32G4 Microcontroller

At the heart of the IOBOX lies the STM32G474MBTx microcontroller from STMicroelectronics based on the high-performance ARM Cortex-M4 core with making it particularly well-suited for real-time signal processing and control applications.

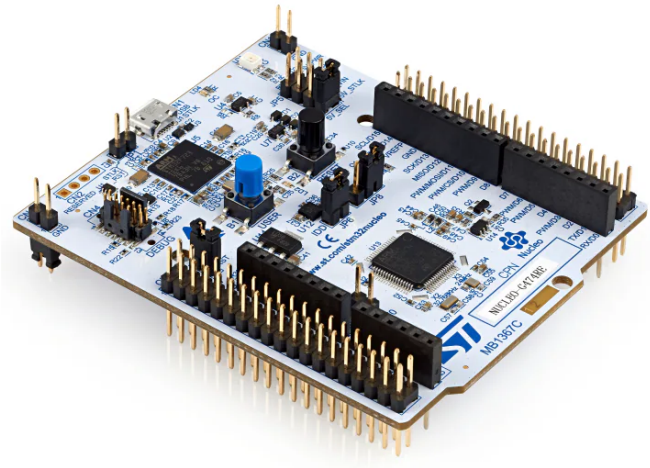

Figure 2.2: STM32G474 Microcontroller (physical package)

The STM32G474 was selected because it provides all the necessary communication protocol interfaces and peripherals that we needed in this project, like the high ADC channel count (five independent ADC units versus two on the F4), its native FDCAN peripheral and its modern HAL ecosystem. Compared to the STM32L4 series, the STM32G474 offers higher clock speed (170 MHz versus 80 MHz) which is necessary for real-time signal processing.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26 × 12-bit; DAC: 7 × 12-bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	–40 °C to +125 °C
Mounting Type	Surface Mount
Package	64-LQFP (10 × 10 mm)
Base Product Number	STM32G474

Within the IOBOX, the STM32G474 acts as the main processing unit for coordinating signal acquisition, processing data and controlling actuators as well as supervising communication with external systems via various industrial protocols.

2.4.2 EEPROM Memory: M95M01-RDW6TP

The IOBOX is equipped with an external EEPROM memory (M95M01-RDW6TP) that comes from STMicroelectronics . This device allows storing the necessary parameters, as well as calibration and logging data for system operation without losing data during power cycling operations.

The M95M01-RDW6TP EEPROM is a 1 Mbit device that operates on the principle of 131,072 × 8 bits . With a speed of 25 ns, and the communication interface is the SPI

bus. It makes an easy integration with STM32G474 . Packaged in an 8 pin SOIC . it provides one milion write cycles , and a period of 40 years of data retention .

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit (131,072 × 8 bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

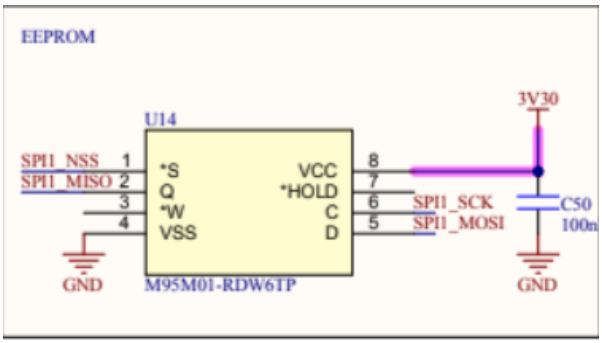

Figure 2.3: M95M01-RDW6TP EEPROM memory chip

2.4.3 Sensors

In this project, we integrated various sensors used for environmental and physical measurements in which our work focused not only on interfacing these sensors with the STM32G474 microcontroller but also on configuring their registers, managing communication protocols and calibrating each one for accurate data acquisition.

ENS210 Sensor

The ENS210 is a digital temperature and humidity sensor from ScioSense that combines both sensing functions within a single chip. The sensor is connected using the I2C bus protocol, facilitating integration into microcontrollers like the STM32G474. The device's high precision and low power dissipation characteristics make it ideal for embedded applications requiring environmental sensing.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	**Specification**
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I^2C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

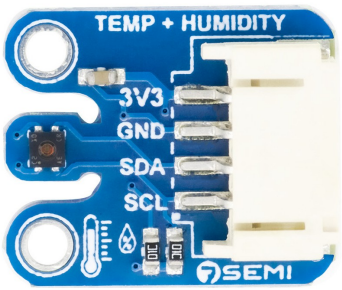

Figure 2.4: ENS210 digital temperature and humidity sensor

OPT3001DNPR Sensor

The OPT3001DNPR is a digital ambient light sensor developed by Texas Instruments. As its name suggests, the sensor measures visible-light intensity outputted in

lux units. This component is suitable for use in automatic brightness control and dimming, display backlighting management, and other similar applications requiring accurate light-level sensing. Communication with the host processor occurs over the I^2C bus. During the development of the current project, integration with the OPT3001DNPR has been performed to acquire ambient light-level measurements. Configuration of measurement modes, threshold detection parameters, and calibration procedures were conducted based on the content of internal registers.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	**Specification**
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I^2C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	–40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)

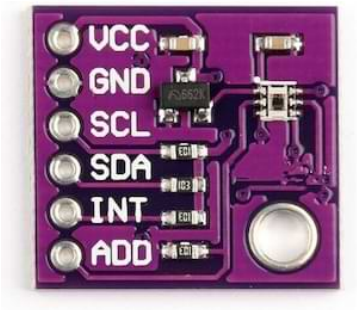

Figure 2.5: OPT3001DNPR ambient light sensor

HTS221TR Sensor

The HTS221TR is a capacitive digital sensor for relative humidity and temperature made by STMicroelectronics, that integrates sensing elements and circuitry for signal

conditioning within its small package and gives calibrated measurement output values using the standard I2C interface. This sensor was designed for low-power applications while maintaining accurate measurement making it the best choice for environmental data monitoring.

In this project, the HTS221TR is used to acquire ambient temperature and relative humidity data using its factory-calibrated coefficients to improve measurement reliability. Communication with the STM32G474 microcontroller is achieved through the I^2C bus, and dedicated software drivers were developed to configure the device and process sensor data.

Table 2.5: Key Characteristics of HTS221TR Sensor

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Temperature + Humidity Sensor
Interface	I^2C (SPI mode not used in this project)
Supply Voltage	1.7 V – 3.6 V
Temperature Range	-40 °C to +120 °C
Temperature Accuracy	±0.5 °C (typ.)
Humidity Range	0 – 100 % RH
Humidity Accuracy	±3.5 % RH (typ.)
Output Resolution	16-bit
Package	HLGA-6 (2 × 2 mm)

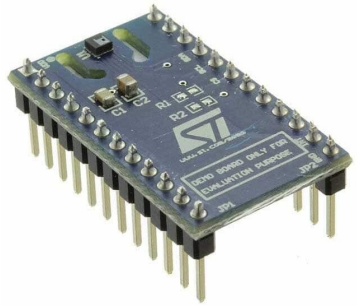

Figure 2.6: HTS221TR temperature and humidity sensor

RS485-to-USB Converter

The RS485-to-USB converter is a compact interface device that bridges the RS485 serial communication standard and the Universal Serial Bus (USB) . Within the IOBOX project, the RS485-to-USB converter played a central role during the development and validation phases. It served as the physical link between the STM32G474MBTx target board and the host computer, enabling the monitoring, testing, and debugging of the Modbus RTU communication layer implemented over the RS485 interface.

Table 2.6: Key Characteristics of the RS485-to-USB Converter

Parameter	**Value**
Type	RS485-to-USB Serial Adapter
Communication Standards	USB 2.0, EIA/TIA-485 (RS485)
USB Interface	USB Type-A / Type-B (Full Speed)
RS485 Interface	Half-duplex, 2-wire differential
Operating Voltage	Bus-powered via USB (5 V)
Maximum Data Rate	Up to 3 Mbps
Isolation	Optional (model-dependent)
OS Compatibility	Windows, Linux, macOS

Figure 2.7: RS485-to-USB converter used for communication testing

2.5 Peripherals

2.5.1 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter (DAC) is one of the analog output peripherals built into the STM32G4 microcontroller. Basically it allows converting digital values

stored in memory into an analog voltage. The resolution is defined by the DAC bit width which allows the generation of precise voltage references or analog waveforms.

The DAC can operate in three main modes: software-triggered mode, timer-triggered mode and DMA-driven mode. In timer-triggered mode, a hardware timer periodically updates the DAC output which keeps the timing consistent. And using the DMA mode reduced the MCU's need to intervene when outputting DAC values which frees it up for other tasks and significantly improving efficiency.

The DAC is used to generate analog reference signals to control actuators and in this project we used two DAC channels that supports software-triggered and timer-triggered output modes.

Figure 2.8 shows an example of DAC signal generation.

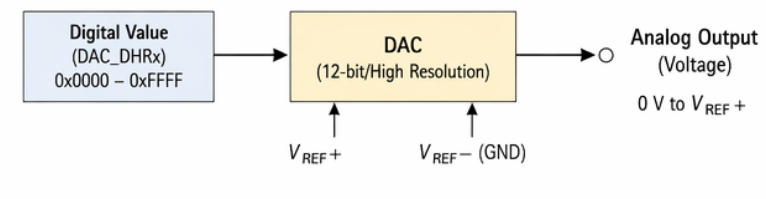

Figure 2.8: Example of analog signal generated by DAC

2.5.2 Analog to Digital Converter (ADC)

Analog-to-Digital Converters (ADCs) represent one of the key peripherals in embedded systems, which convert analog continuous-time signals into discrete digital equivalents for further processing by the microcontroller. Such conversions are critical for sensing and analyzing physical phenomena such as temperature, pressure, and current.

In the IOBOX project, ADCs serve for acquisition of sensor readings and data from additional modules connected to the system. The STM32G474 MCU features several ADC channels, providing simultaneous sampling of various inputs. This feature proves useful when handling multiple simultaneous signals typical of industrial use cases.

ADC operation involves a sequence of steps performed for each conversion cycle. Firstly, the analog signal is sampled periodically based on the sampling frequency. Next, each sample point is converted to a corresponding digital representation using the ADC resolution (e.g., 12-bit and 16-bit). Thus obtained digital word corresponds to the amplitude of the input analog signal at that moment in time. The overall conversion accuracy largely depends on parameters such as resolution, sampling frequency, and stability of the reference voltage.

Key properties of ADCs include:

- **Resolution**: defined by the amount of discrete levels supported.
- **Sampling rate**: Specifies how often the analog value will be sampled, thus impacting the accuracy of dynamic signals.
- **Reference voltage**: determines the maximum value of the input signal to be measured, providing appropriate scaling of the output result.
- **Input channels**:Multiple input channels can measure several inputs from various sensors in parallel.
- **Conversion modes**:Several conversion modes include single-shot, continuous, and DMA-based conversions, allowing exible embedding in the firmware.

ADCs in the IO-Box collect raw analog data like voltage, current, or any other physical quantity sensed by the attached sensors. The collected data is processed in the firmware layer to extract valuable information, take certain actions based on this data, or send it further for analysis by supervisory systems.

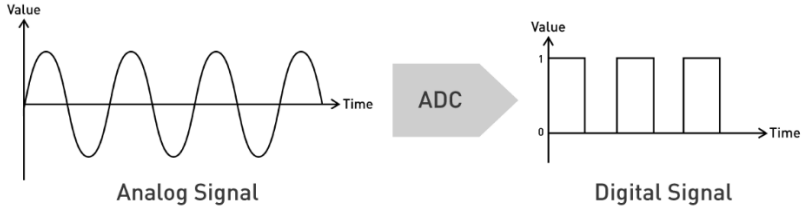

Figure 2.9: Basic ADC conversion from analog input to digital output

2.5.3 Timers

The timer peripheral module in the STM32G474 microcontroller is an essential component for precise time base generation, counting events, and measuring signals. It works by incrementing/decrementing a counter register based on a clock input. Actions can be initiated whenever certain thresholds are met in the counter value.

Key features of timers include:

- **Timer Functions:**: Time Base Generation refers to generating accurate delays or repetitive events.
- **Input Capture** : Involves measuring properties of external signals, including their frequencies or pulse widths.

- **Output Compare**: consists of triggering events based on matching values of the counter register with reference values.
- **Synchronization**: includes synchronizing several timer modules to achieve more complicated control applications.

Timers are fundamental components in many applications involving scheduling of operations, processing sensor measurements, and controlling peripherals requiring accurate timing.

2.5.4 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) refers to a method for representing analog values as digital signals based on the variation in the duty cycle within a periodic wave signal. In embedded systems applications, PWM plays an essential role in controlling DC motors, LED brightness adjustment, and power conversion.

On the STM32G474 platform, PWM generation occurs via the timer peripheral modules. A timer counter increases its value until reaching the limit value set in the auto-reload register (ARR). The compare register (CCR) sets the switchpoint of the PWM output signal. The relationship between the pulse duration and the complete period determines the duty cycle percentage. Through ARR and CCR register manipulation, precise frequency and duty cycle control become possible.

Key characteristics of PWM include:

- **Frequency**: Determined by prescaler and ARR configuration.
- **Duty Cycle**: Defined by CCR relative to ARR.
- **Resolution**: Limited by timer bit width (typically 16-bit).
- **Complementary Outputs**: Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications**: Motor drivers, LED dimming, switching converters, and DAC emulation.

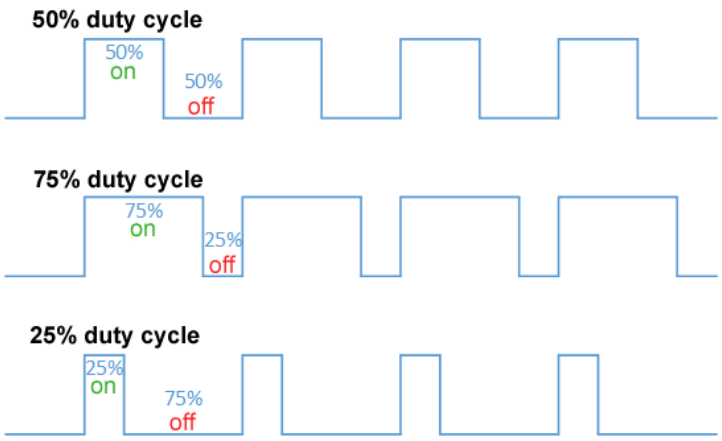

Figure 2.10: Illustration of PWM signals with different duty cycles

2.6 Communication Protocols

2.6.1 I2C Protocol

Inter-Integrated Circuit (I2C) is a synchronous, half-duplex serial communication protocol widely used for communication between integrated circuits over short distances. It relies on two bidirectional lines: the Serial Data Line (SDA) and the Serial Clock Line (SCL), both requiring pull-up resistors. The protocol follows a master-slave architecture, where the master initiates communication and generates the clock signal.

Each device connected to the bus is identified by a unique address. Data is then exchanged one byte at a time, with each byte followed by an acknowledgment bit (ACK/NACK) from the receiving device. The communication is terminated by a STOP condition. I2C supports different speed modes, including Standard Mode (100 kHz), Fast Mode (400 kHz), and higher-speed variants.

The I2C frame structure is illustrated in Figure 2.11.

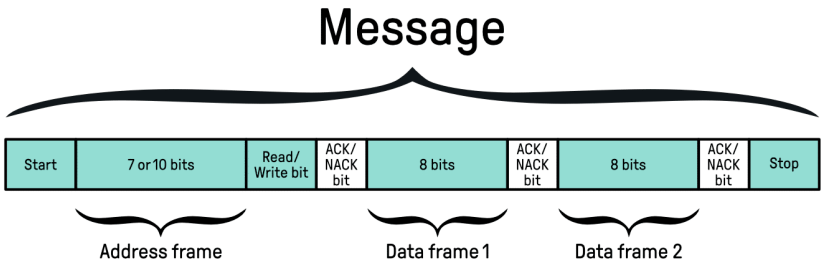

Figure 2.11: I2C frame format

In this project, the I2C interface of the STM32G474MBTx is configured using the

HAL library generated by STM32CubeMX. It is used to communicate with external peripherals such as sensors and low-speed devices. The implementation relies on interrupt or polling-based data transfers depending on timing constraints. Particular attention is given to bus stability, including proper pull-up resistor sizing and handling of potential communication errors such as bus busy states or missing acknowledgments.

As shown in Figure 2.12, the I2C bus allows multiple slave devices.

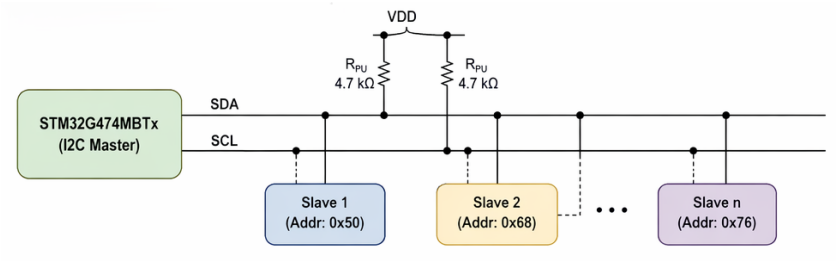

Figure 2.12: I2C communication bus architecture

2.6.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master–slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IOBOX project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. The SPI protocol employs synchronous communication controlled by the clock signal (SCLK) to guarantee predictable timing performance. A basic transaction involving SPI communication involves the activation of the chip select (CS) line corresponding to a specific slave device, followed by the generation of the clock signal, transferring bits sequentially using MOSI and reading bits via MISO, sampling the data based on the mode selected, and terminating the connection by deactivating the CS line. As a result, bidirectional simultaneous data transfer is made possible. Four different operational modes are available with SPI communication, designated as Modes 0 to 3, based on the values of the CPOL and CPHA parameters. Such flexibility makes the protocol compatible with various types of peripherals. Main features of SPI

communication include high-speed data transfer (several MHz), straightforward cabling (four wires: MOSI, MISO, SCLK, CS), and scalable implementation using several slaves.

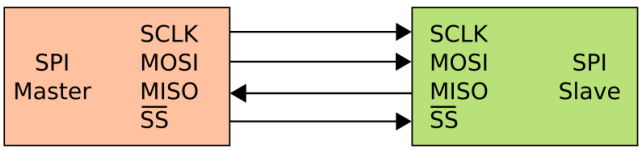

Figure 2.13: Basic SPI communication between master and slave devices

2.6.3 UART Protocol

UART is an asynchronous full-duplex serial communication protocol used to exchange data between devices without the need for a shared clock signal. Data is transmitted over two lines: Transmit (TX) and Receive (RX) and sometimes labeled as TXD and RXD depending on the datasheet. Synchronizing between devices by configuring identical communication parameters like baud rate, number of data bits, parity bit setting and stop bit configuration.

Each data frame typically consists of a start bit (always low), followed by a configurable number of data bits (8 in our case), an optional parity bit for error detection and one or sometimes two stop bits. UART has no clock line which keeps the hardware simple, but, it also means both sides have to already agree on the same baud rate beforehand, otherwise, if they drift even slightly, the bits will get misread.

UART is used to communicate between the STM32 microcontroller and external systems such as a host computer or other embedded modules. It plays a key role in debugging, logging, and real-time data exchange. The implementation uses the STM32 HAL drivers with support for interrupt-driven and DMA-based transmission to ensure efficient and non-blocking communication. Error handling mechanisms, including overrun, framing, and noise errors, are also considered to improve communication reliability.

Figure 2.14 shows the UART frame structure.

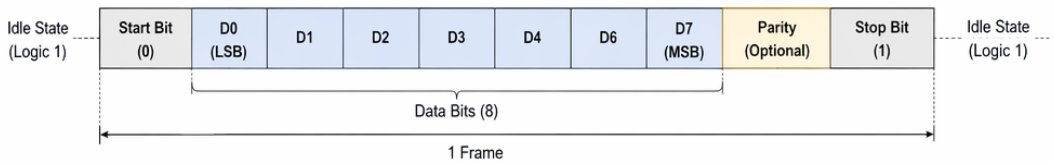

Figure 2.14: UART frame format

2.6.4 RS485 Communication

RS485 is a robust differential serial communication standard widely used in industrial environments due to its high noise immunity and long-distance capabilities. Unlike single-ended UART communication, RS485 transmits data over a differential pair of lines (A and B), where the information is encoded as a voltage difference between the two conductors. This approach significantly reduces susceptibility to electromagnetic interference and ground potential differences.

The RS485 standard supports half-duplex communication in multi-drop configurations, allowing multiple nodes to share the same bus. However, only one device can transmit at a time, which requires proper bus arbitration at the software level or through protocol design.

In this project, RS485 serves as the physical layer for Modbus RTU communication. A dedicated UART peripheral (USART3) is used with an external RS485 transceiver to convert TTL logic levels to differential signals. Direction control is managed in software, as described in Chapter 3.

Before transmission, the microcontroller sets the DE pin high to enable the driver stage. After sending the data frame, the DE pin is reset to return the transceiver to reception mode. This timing control is critical to avoid bus contention. The communication is typically used for reliable data exchange in noisy environments, especially when longer cable lengths are required compared to standard UART.

Figure 2.15 illustrates a typical RS485 bus configuration.

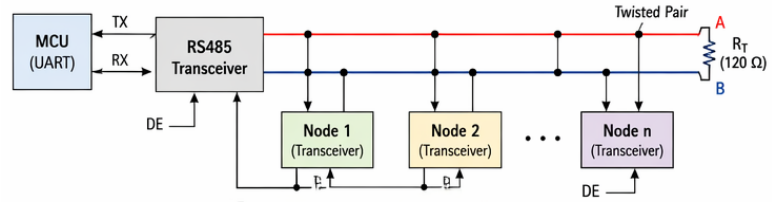

Figure 2.15: RS485 differential bus with multi-node configuration

2.6.5 Modbus Protocol

Modbus is one of the most widely used industrial communication protocols for exchanging data between electronic devices. Originally developed by Modicon in 1979, it follows a client-server (master-slave) architecture and enables communication between programmable logic controllers (PLCs), sensors, actuators, Human-Machine Interfaces (HMIs), and embedded systems.

Modbus defines a standardized message structure that allows devices from different manufacturers to communicate using a common protocol. Two main variants are commonly used: Modbus RTU, which operates over serial communication interfaces such as RS232 and RS485, and Modbus TCP, which operates over Ethernet networks.

In the IOBOX project, Modbus RTU is implemented over the RS485 physical layer to ensure reliable communication in industrial environments. The STM32G474 microcontroller acts as either a Modbus master or slave depending on the application requirements. Through Modbus registers, external systems can access sensor measurements, configuration parameters, status information, and control commands.

In this project, the IOBox operates as a Modbus RTU slave at address 0x01, communicating over RS485 at 19,200 bps. The slave implementation supports function codes FC01 through FC06, FC15, and FC16. Input registers expose real-time sensor measurements (temperature, humidity, illuminance) to any connected Modbus master, and holding registers allow actuator control. The implementation details are described in Chapter 3.

The Modbus RTU frame consists of four main fields:

- **Slave Address**: Identifies the target device.
- **Function Code**: Specifies the requested operation.
- **Data Field**: Contains transmitted data or register information.
- **CRC Field**: Provides error detection for frame integrity.

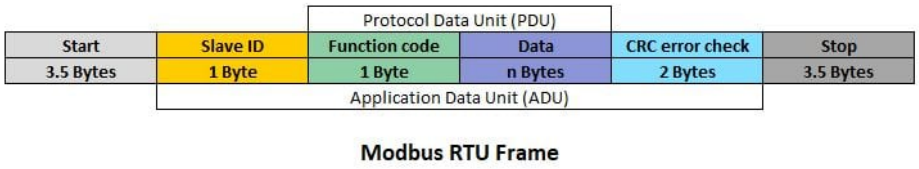

Figure 2.16: Typical Modbus RTU frame structure

To make sure that the implementation is compatible with standard industrial devices, the implementation supports a subset of Modbus function codes, classified according to their role in data access and control:

Table 2.7: Supported Modbus Function Codes in the IOBOX Platform

Function Code	**Name**	**Description**
FC01	Read Coils	Reads digital output states (ON/OFF status)
FC02	Read Discrete Inputs	Reads digital input signals from sensors or switches
FC03	Read Holding Registers	Reads configuration and control registers (16-bit)
FC04	Read Input Registers	Reads real-time measurement data (sensor values)
FC05	Write Single Coil	Controls a single digital output (ON/OFF command)
FC06	Write Single Register	Writes a single 16-bit configuration or control value
FC15	Write Multiple Coils	Updates multiple digital outputs in one request
FC16	Write Multiple Registers	Writes multiple registers for bulk configuration/control

This classification enables structured interaction with the IOBOX memory map, separating real-time acquisition data (input registers) from controllable parameters (holding registers). It also ensures interoperability with a wide range of Modbus-compatible supervisory systems such as SCADA platforms and PLCs.

2.6.6 Controller Area Network (CAN)

The Controller Area Network (CAN) is a dependable serial communication protocol primarily designed for automotive applications, but today extensively utilized in industry and embedded computing systems. Unlike traditional network protocols, it does not require a host node since CAN facilitates data communication among various devices connected to the same network. For the IOBOX project, CAN communication technology has been selected because of its high reliability and fault tolerance properties, essential for communication with industrial equipment and sensors. The capability to incorporate many nodes into the same CAN bus network makes it appropriate for

applications involving multiple devices transmitting information simultaneously. Communication in CAN occurs according to a messaging protocol. Rather than addressing particular nodes, every transmitted message contains an identifier field specifying its priority and type of content. As a result, all nodes receive a message and determine if the incoming information needs further processing. Arbitration allows two nodes communicating simultaneously to resolve any potential conflicts: the node transmitting a message with a higher priority (a lower identifier number) wins, and another node retries sending a message later after the bus becomes free. Key features of the CAN protocol:

- **Multi-master support**: Communication can be initiated by any node whenever the bus becomes available.
- **Error detection and correction**: Various methods, including CRC, acknowledgment bits, and error frames, guarantee data reliability.
- **Priority-based arbitration**: Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability**: Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed**: Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

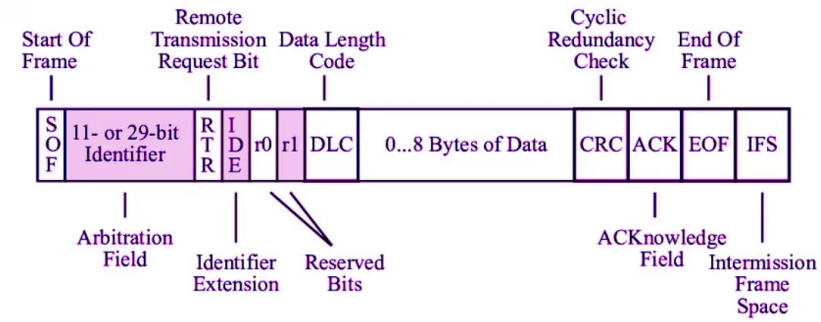

Figure 2.17: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

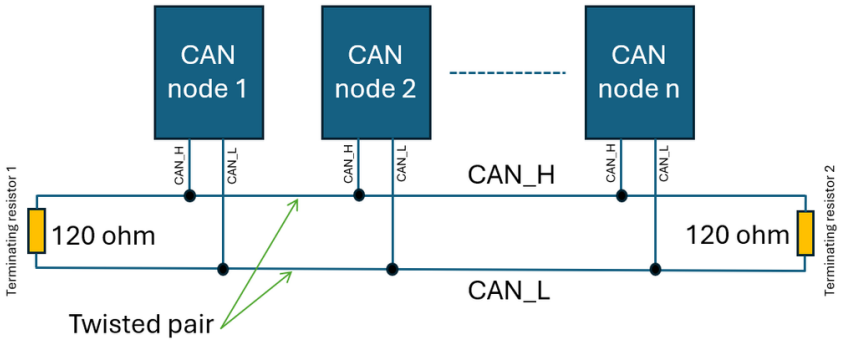

Figure 2.18: Basic CAN bus communication with multiple nodes

2.6.7 LIN Protocol

Local Interconnect Network (LIN) is a low-cost serial communication protocol designed for distributed control systems, particularly in automotive applications. It operates on a single-wire physical layer and follows a strict master-slave architecture where the master node controls the entire bus communication schedule.

Unlike event-driven protocols, LIN is time-triggered which means communication is organized into predefined frames sent at fixed intervals. Each frame consists of a header transmitted by the master and a response transmitted by a slave node. The header includes a break field (used to signal the start of a frame), a synchronization byte, and an identifier field that determines which node should respond.

The data response typically contains up to 8 bytes, followed by a checksum used for error detection. LIN achieves deterministic timing by enforcing strict scheduling, making it suitable for non-critical but predictable control tasks.

LIN communication was implemented using the UART peripheral configured in LIN mode. The hardware support simplifies break detection and synchronization field handling, reducing software complexity. The system can therefore reliably detect frame boundaries and process incoming data with minimal CPU overhead.

Figure 2.19 shows the LIN frame structure.

2.7 Software tools

2.7.1 Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight source code editor develpoed by Microsoft, supporting multiple programming languages and providing fully customizable features such as Git integration, integrated terminal and debugging tools[3].

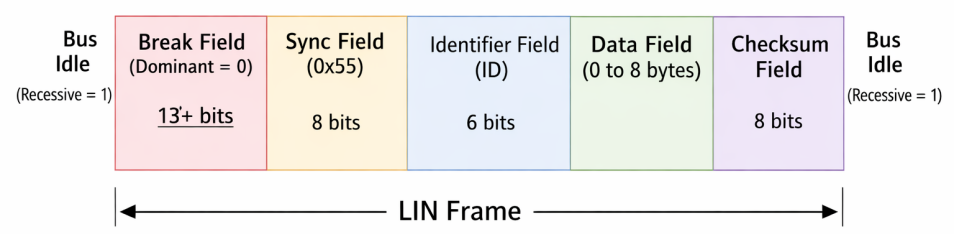

Figure 2.19: LIN frame structure

In this project, VS Code was used for documentation and Git repository version control using Git Graph extension for a clear visualization of commit history, branch management, and tracking of code changes across the project's development.

Figure 2.20: Visual Studio Code interface

2.7.2 IAR Embedded Workbench

IAR Embedded Workbench (IAR EW) offers a fully integrated development environment (IDE) with highly optimized C/C++ compiler, advanced debugging capabilities, and efficient project management tools, eliminating the need for multiple third-party tools[4].

In this project, IAR EW was used for firmware development, compilation, and debugging. Its powerful optimization features contributed to generating efficient and reliable embedded code, which is critical for real-time applications and resource-constrained systems.

Figure 2.21: IAR Embedded Workbench IDE

2.7.3 STM32CubeMX

STM32CubeMX is a graphical configuration tool developed by STMicroelectronics for STM32 microcontrollers. It allows developers to easily configure peripherals, clock settings, and pin assignments through an intuitive interface. The tool also generates initialization code based on the selected configuration, significantly reducing development time and minimizing configuration errors[5].

In this project, STM32CubeMX was used in a limited capacity: it was invoked once at the beginning of the project to generate the initial project skeleton, configure the system clock (HSI PLL at 170 MHz), and enable the Serial Wire Debug interface. The peripheral initialization functions that CubeMX would normally generate were intentionally not used. Instead, all peripheral drivers were developed as handwritten BSP (Board Support Package) modules, which are described in detail in Chapter 3. CubeMX thus served exclusively as a project scaffolding and clock configuration tool.

Figure 2.22: STM32CubeMX configuration interface

2.7.4 GitLab

GitLab was used as the version control and collaboration platform throughout the project. The repository was organized by module (BSP, Managers, Application), with branches used to isolate feature development and avoid conflicts during parallel work. Commit history served as a development log, and merge requests were used to review code before integrating changes into the main branch.

Figure 2.23: GitLab logo

Modbus Poll

Modbus Poll is a Windows-based Modbus master simulation and diagnostic tool widely used in industrial embedded development. It allows engineers to send Modbus requests, read and write registers and coils, and monitor the raw communication frames. In the context of the IOBOX project, Modbus Poll was used as the primary host-side interface for validating the Modbus RTU slave implementation running on the STM32G474MBTx. Connected to the target board through the RS485-to-USB converter.

Figure 2.24: Modbus Poll logo

2.7.5 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be

dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

CHAPTER 3

DESIGN AND PRACTICAL IMPLEMENTATION

3.1 Introduction

In this chapter, we present the design and practical implementation of the IOBOX platform. Building on the requirements and hardware foundations introduced in the previous chapters, we describe the key design choices and software architecture that transform these elements into a functional embedded system. This chapter provides a brief overview of the firmware implementation and the main technical decisions that ensure modularity and scalability for industrial applications.

3.2 Software Architecture

The firmware of the IOBOX platform follows a structured four-layer architecture. At the lowest level, the hardware abstraction layer (HAL) that provides direct communication with the hardware. Above it, the Board Support Package (BSP) handles board-specific configuration including bus initialization and public API. The third layer, a Manager layer encapsulating peripheral control and communication protocol logic, as a higher-level API. At the top, an application layer that implements the system's business logic and task management and execution. This strict architecture enforces a clear separation of the code layers ensuring that each layer can only access the APIs of the layer beneath it.

To ensure that only the peripherals and protocols necessary for a particular deployment are compiled into the firmware binary, all hardware features are enabled at compile time using feature-enable macros defined in the application configuration file,

`feature_config.h`. These preprocessor flags control the inclusion of each hardware feature, allowing the firmware to be tailored to specific application requirements.

All application modules share a single global structure named `SystemBoard`, declared in `feature_config.h`. Acting as a centralized data repository, it enables data exchange between modules in a blackboard-like manner, where each module updates its latest information and retrieves data produced by others. The structure consolidates real-time sensor measurements, CAN frame data, and system status information, and is accessed directly by the Manager and Application layers during each execution cycle.

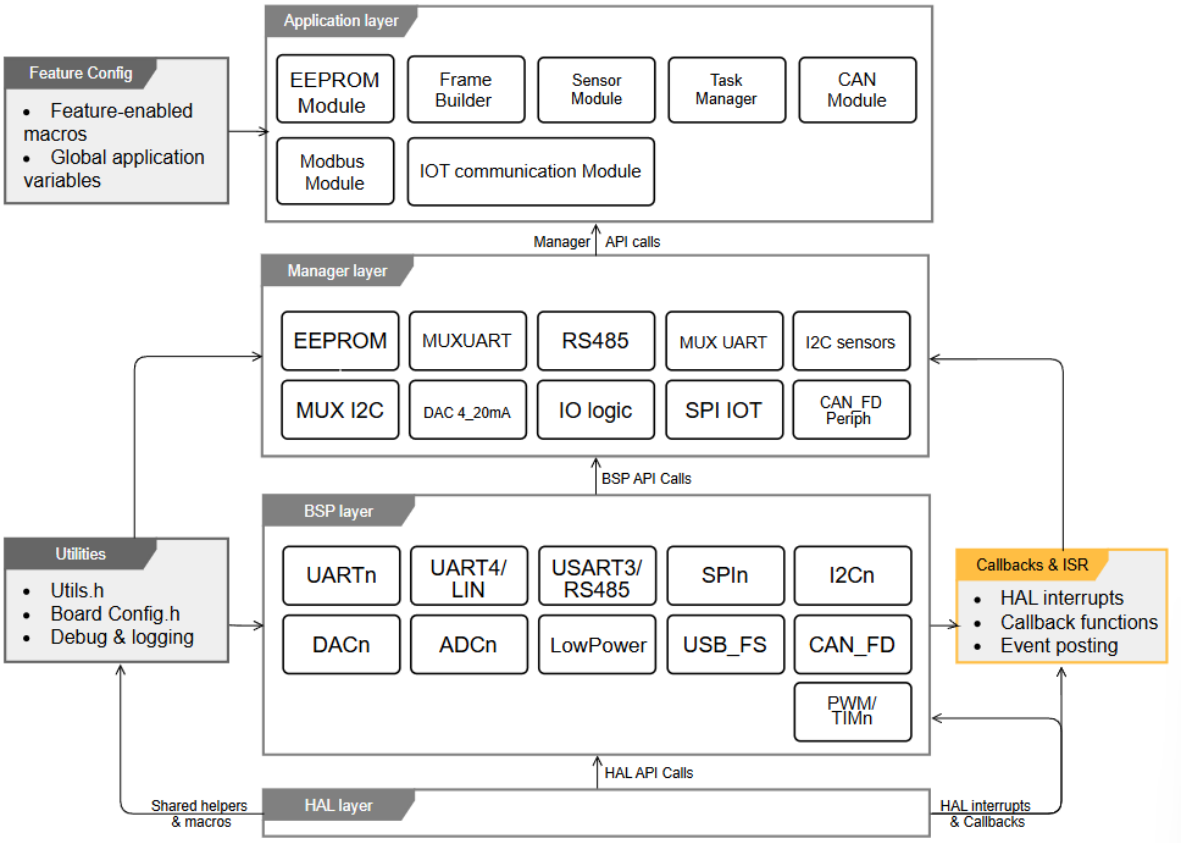

Figure 3.1: Layered software architecture of the IOBOX platform

The following subsections will describe each layer in detail, from the lowest software layer that interfaces directly with the hardware to the highest layer that implements application logic.

3.2.1 Board Support Package Layer

At the bottom of the firmware stack lies the Board Support Package (BSP). The layer is organized into separate driver modules, each responsible for the initialization, configuration, and data transfer operations of a specific hardware interface. All register-level and HAL-level operations are confined to this layer, allowing the upper layers to remain independent of hardware-specific details and interact with peripherals exclusively through the BSP public API.

Each BSP module follows a consistent structure: a header file located under `BSP/Inc/` declares the public API, while the corresponding source file under `BSP/Src/` contains the implementation.

The platform exposes BSP modules for the following peripheral categories:

- For serial communication, `BSP_RS485` implements the half-duplex differential bus used by Modbus RTU; `BSP_LIN` handles the single-wire LIN bus; `BSP_USART1`, `BSP_USART2`, `BSP_UART5`, and `BSP_LPUART1` provide general-purpose asynchronous communication; and `BSP_UART_MUX` manages the software-controlled UART channel multiplexer.
- High-speed synchronous communication is handled by three SPI drivers: `BSP_SPI1`, `BSP_SPI2`, and `BSP_SPI4`. Sensor and peripheral expansion buses rely on `BSP_I2C2` and `BSP_I2C3`, complemented by `BSP_I2C_MUX`, which controls the TCA9548A eight-channel I^2C multiplexer.
- The CAN interface is abstracted through `CAN1_BSP`, while digital input/output functionality is provided by `BSP_DigitalIO`.
- Analog functionality is split between ADC and DAC drivers. Four ADC modules (`ADC1_BSP`, `ADC3_BSP`, `ADC4_BSP`, and `ADC5_BSP`) expose acquisition functions for active conversion channels, while `BSP_DAC1` and `BSP_DAC2` provide analog voltage generation.
- Four timer drivers (`TIM1_BSP` through `TIM4_BSP`) manage time-base generation and PWM functionality.
- A dedicated low-power management module, `BSP_LowPower`, groups the routines responsible for power mode transitions.

This organization simplifies platform migration. When porting the firmware to another STM32 family, most hardware-dependent modifications remain confined to the BSP layer, while the Manager and Application layers can generally be reused without significant changes.

3.2.2 Manager Layer

Three sensor manager modules were developed, one per physical sensor, each handling I^2C multiplexer channel selection, register-level communication, and raw-to-physical value conversion.

HTS221TR Manager

The HTS221TR manager drives the STMicroelectronics capacitive humidity and temperature sensor connected to I^2C multiplexer channel 0. Subsequent acquisitions apply linear interpolation between these reference points to convert the 16-bit raw ADC output into calibrated temperature in degrees Celsius and relative humidity in percent, as shown in Equations 3.1 and 3.2.

$$\%\mathrm{RH} = H_{0,\mathrm{rH}} + \frac{(H_{1,\mathrm{rH}} - H_{0,\mathrm{rH}}) \cdot (\mathrm{raw} - H_{0,T_0})}{H_{1,T_0} - H_{0,T_0}} \tag{3.1}$$

$$T\,[^\circ\mathrm{C}] = T_{0,\mathrm{degC}} + \frac{(T_{1,\mathrm{degC}} - T_{0,\mathrm{degC}}) \cdot (\mathrm{raw} - T_{0,\mathrm{out}})}{T_{1,\mathrm{out}} - T_{0,\mathrm{out}}} \tag{3.2}$$

Data readiness is verified by polling the sensor's status register before each acquisition. If the data-ready flag is not asserted within a configurable retry count, the acquisition routine returns a timeout status without blocking the system further.

ENS210 Manager

The ENS210 manager interfaces with the sensors combined temperature and humidity measurements on multiplexer channel 2. The module operates in single-shot mode: a measurement is triggered by writing to the sensor's start register, a fixed 10-millisecond settling delay is applied, and the 16-bit raw temperature and humidity registers are then read. Conversion follows the datasheet-specified formulae: temperature is obtained as $T_K = \mathrm{raw}/64$, then converted to Celsius by subtracting 273.15; relative humidity is obtained as $\%\mathrm{RH} = \mathrm{raw}/512$.

OPT3001DNPR Manager

The OPT3001DNPR manager drives the Texas Instruments ambient light sensor on multiplexer channel 1. The sensor is configured at initialisation by writing a control word to its configuration register, selecting continuous conversion mode, automatic full-scale range, and an 800-millisecond conversion time. The 16-bit result register encodes the measurement as a four-bit binary exponent and a twelve-bit mantissa; the illuminance in lux is recovered by the expression $E_{\mathrm{lux}} = \mathrm{mantissa} \times 0.01 \times 2^{\mathrm{exponent}}$.

In all three managers, the I^2C multiplexer channel is selected at the start of each transaction and disabled upon completion, preventing bus contention between sensors sharing the same physical I^2C bus.

Modbus RTU Manager

The Modbus RTU manager provides both master and slave roles over the RS485 physical layer and is entirely self-contained within a single manager module. In the current platform configuration, only slave mode is active which relies on interrupt-driven reception to process incoming requests. Eight function codes are supported: FC01 to FC06, FC15 and FC16. Frames addressed to wrong slave addresses are silently discarded, frames with invalid CRC are dropped without response, according to the Modbus RTU specification. Following each processed request, the receiver is immediately re-armed to avoid missing subsequent requests.

Table 3.1 summarizes the Modbus RTU function codes implemented in the system, covering both coil and register-level read/write operations used by the slave device.

Function Code	**Name**	**Description**
FC01	Read Coils	Reads the ON/OFF status of discrete output coils from the slave device.
FC02	Read Discrete Inputs	Reads the status of discrete input contacts from the slave device.
FC03	Read Holding Registers	Reads one or more 16-bit holding registers used for general-purpose data storage.
FC04	Read Input Registers	Reads 16-bit input registers representing analog or measured data.
FC05	Write Single Coil	Writes a single ON/OFF value to a discrete output coil.
FC06	Write Single Register	Writes a single 16-bit value to a holding register.
FC15	Write Multiple Coils	Writes multiple ON/OFF values to consecutive output coils in a single request.
FC16	Write Multiple Registers	Writes multiple 16-bit values to consecutive holding registers.

Table 3.1: Modbus RTU Function Codes Used in the System

CAN Manager

The CAN manager deliver a middle layer capturing frames as they arrive and storing them until the application layer retrieves them ensuring no frame is lost , making it immediately available to be processed and stored in the eeprom .

EEPROM Manager

The EEPROM Manager, implemented in `EEPROM_manager.c` , uses the M95M01-RDW6TP serial EEPROM via the SPI BSP layer, creating a store and load fonctions that allow the Application layer to conserve the `DataFrame` structure across power cycles and prevent data loss .

3.2.3 Filter Manager

Industrial sensor signals are exposed to electrical interference, thermal gradients, and inherent sensor noise introduce unwanted deviation into raw measurements . if left unprocessed it will degrade the accuracy of the data delivered to the application layer. To solve this problem the IOBOX firmware provides a dedicated, generic signal processing module implemented in `Filter_Manager.c` and `Filter_Manager.h`.

The seven filter algorithms provided by the module are described individually below.

Exponential Moving Average Filter

Purpose. The Exponential Moving Average (EMA) is a low pass filter designed to smooth noisy sensor readings by ensuring that all samples contribute to the analysis while giving more priority to recent observations.

Main Equation.

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1] \tag{3.3}$$

Parameters.

- $x[n]$ is the current raw sample .
- $y[n - 1]$ is the previous filtered output .
- $\alpha \in (0, 1]$ is the smoothing factor.

Simple Moving Average Filter

Purpose. The Simple Moving Average (SMA) is a low pass filter that stabilizes a signal by calculating the arithmetic mean of the most recent samples.

Main Equation.

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{3.4}$$

Parameters.

- N is the window size.
- $x[n-k]$ represents the sample collected .
- k steps before the current one.

First-Order IIR High-Pass Filter

Purpose. The first order IIR high pass filter removes slowly changing signal components and static offsets from a signal keeping only its dynamic .

Main Equation.

$$y[n] = \alpha \cdot \big(y[n-1] + x[n] - x[n-1]\big) \tag{3.5}$$

Parameters.

- $x[n]$ is the current input .
- $x[n-1]$ is the previous input .
- $y[n-1]$ is the previous output .
- $\alpha \in (0,\, 1]$ is the filter coefficient.

Simple Difference High-Pass Filter

Purpose. The simple difference filter is a high-pass filter . It extracts the sample to sample variation of a signal and cumpute an approximation of its rate of change.

Main Equation.

$$y[n] = x[n] - x[n-1] \tag{3.6}$$

Parameters.

- $x[n-1]$ is the previous input value .
- $x[n]$ is the current input value .
- $y[n]$ is the current output value .

SMA Subtraction High-Pass Filter

Purpose. The SMA subtraction filter is a high pass filter that extracts rapid signal variations by removing the local average from each measuruments.

Main Equation.

$$y[n] = x[n] - \frac{1}{N}\sum_{k=0}^{N-1} x[n-k] \tag{3.7}$$

Parameters.

- $y[n]$ is the filtered output signal .
- $x[n]$ is he current input signal .
- N is the window size .
- k is the summation index .

One State Position Kalman Filter

Purpose. The one state Kalman filter provides a changing scalar quantity by combining model predictions with noisy measurements to improve accuracy.

Main Equations.

$$K = \frac{P+Q}{P+Q+R} \tag{3.8}$$

$$\hat{x} = \hat{x} + K \cdot (z - \hat{x}) \tag{3.9}$$

$$P = (1-K)\,(P+Q) \tag{3.10}$$

Parameters.

- Q is the process noise variance
- R is the measurement noise variance .

- $\hat{x}$ is the current state estimate .
- P is the estimation error covariance .
- z is the new measurement .
- K is the Kalman gain computed at each step.

Two-State Constant-Velocity Kalman Filter

Purpose. The two-state Kalman filter improves the position model by additionally tracking the rate of change (velocity) of the measured quantity.

Main Equation. The state prediction is:

$$\begin{bmatrix} \hat{x}_{\text{pos}} \\ \hat{x}_{\text{vel}} \end{bmatrix}_n^- = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{x}_{\text{pos}} \\ \hat{x}_{\text{vel}} \end{bmatrix}_{n-1} \tag{3.11}$$

Parameters.

- $\hat{x}_{\text{pos}}$: estimated position of the system.
- $\hat{x}_{\text{vel}}$: estimated velocity of the system.
- $\begin{bmatrix} \hat{x}_{\text{pos}} \\ \hat{x}_{\text{vel}} \end{bmatrix}$: state vector containing the estimated position and velocity.
- n: current time step.
- $n-1$: previous time step.

Filter Manager: Architectural Summary

The Filter Manager integrates seven algorithms covering the signal processing needs within the IOBOX firmware. Low-pass filters (EMA, SMA) ensure noise reduction and signal stabilization, while high-pass filters (IIR High-Pass, Simple Difference, SMA Subtraction) extarcts rapid changes and remove drift. Kalman filters improve measurement accuracy by combining sensor readings with model predictions . while the two-state version also estimates the rate of change of the measured quantity. the available filters offer a filtering effectiveness which allows the Application layer to use the most suitable solution for each signal.

3.2.4 Application Layer

The application layer is the highest firmware layer and acts as the brain of the system. It does not interact with hardware directly; instead, it coordinates the manager modules, handles data exchange, and constructs the final output frame. Each system functionality is implemented through a dedicated application module built on top of manager APIs and responsible for a specific part of the system logic.

For this project, four main modules were developed: a sensor module for data acquisition and processing, a Modbus RTU slave module for communication handling, a frame builder for UART-based communication, and a task manager to coordinate module execution. Each module is enabled through a compile-time feature macro defined in `feature_config.h`, ensuring that only the required features are included and executed in the final firmware deployment.

Figure 3.2 illustrates the execution workflow of the application layer. It shows the initialization phase, the cyclic cooperative task manager, and the sequential execution of core modules including sensor acquisition, CAN monitoring, EEPROM storage, and frame construction. The diagram also highlights the event-driven nature of the Modbus RTU slave, which is triggered by interrupt-based request detection while response handling is performed within the scheduled task context.

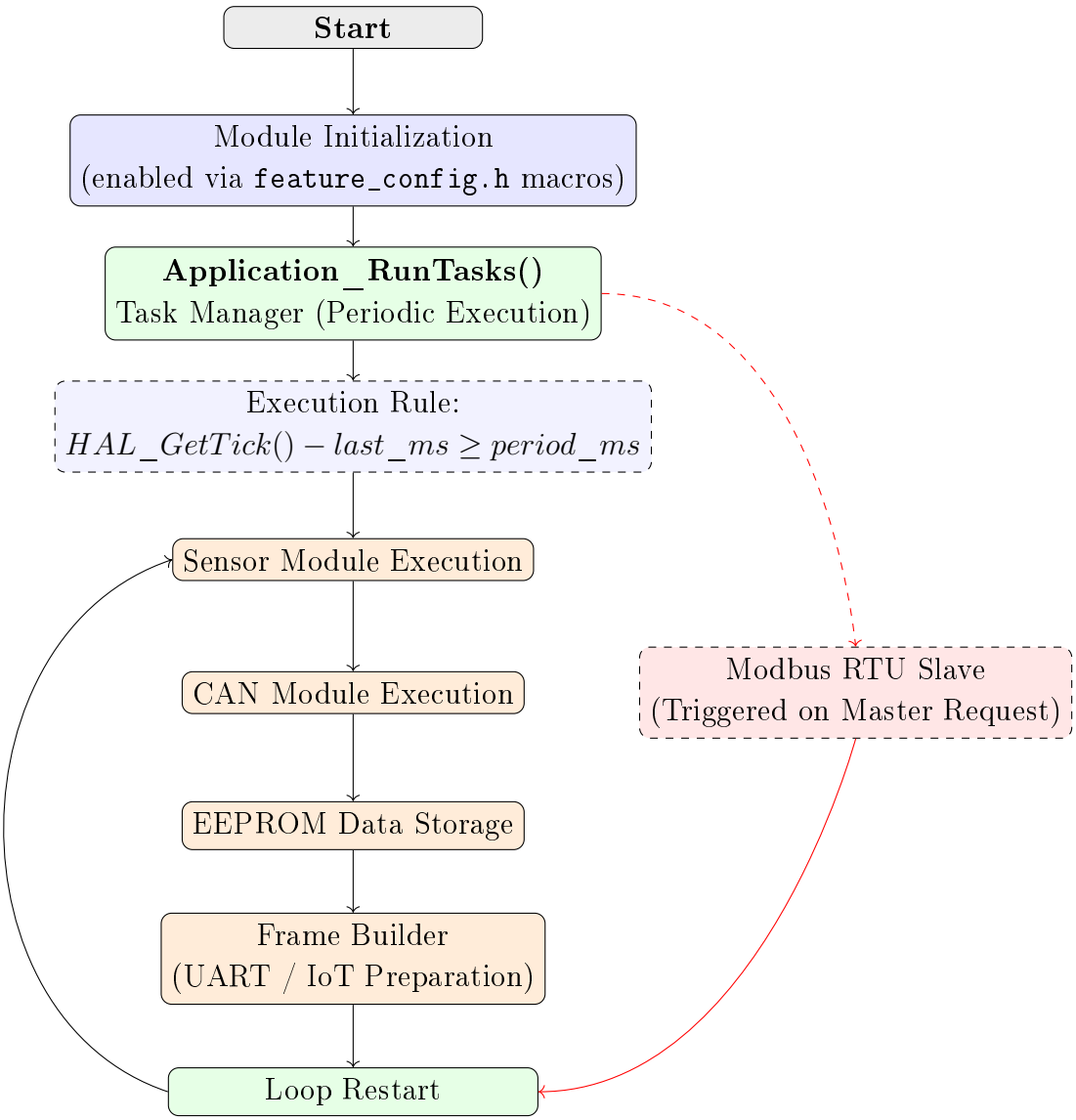

Figure 3.2: Application Layer Workflow diagram

Cooperative Task manager

For this project, we used a lightweight cooperative task manager instead of the traditional RTOS approach. The task manager is implemented as a statically allocated array of task entry structures, where each element is initialized with a timestamp representing the last execution time, an execution period in milliseconds, and a function pointer.

To run these tasks, the dispatcher function `Application_RunTasks()` iterates over the array elements during each iteration of the main loop and invokes each module task whenever the elapsed time since its last execution equals or exceeds its configured period.

Currently, six tasks are registered in the current system configuration, as shown in Table 3.2. Most module task functions are designed to be non-blocking, with some relying on interrupts. Each task returns immediately if no new data is available or if their internal period has not elapsed, preventing any single task from monopolizing MCU time or delaying the execution of other tasks.

Table 3.2: Registered tasks in the cooperative scheduler

Task Function	**Period (ms)**	**Responsibility**
`Sensor_Module_Task`	10	Poll all enabled sensors
`CAN_Module_Run`	10	Drain CAN FIFO and update `sysboard.can`
`Modbus_Module_Poll`	20	Process incoming Modbus RTU frames
`EEPROM_StoreFrame`	1000	Persist current `SystemBoard` to EEPROM
`EEPROM_LoadTask`	1000	Reload last stored frame from EEPROM
`FrameBuilder_Task`	1000	Assemble and finalise the output `IoFrame`

Sensor Module

The Sensor Module provides a unified acquisition interface for all enabled sensors. It maintains per-sensor timestamp variables and determines whether each sensor's configured sampling period has elapsed by comparing the current system tick with the stored timestamps during each `SensorModule_Task()` execution.

When a period expires, the corresponding sensor readings are immediately updated in the `Sysboard` global structure, and the associated manager function is invoked.

The sensor sampling periods are defined in `feature_config.h`: 1000 ms for the `HTS221TR`, 1000 ms for the `ENS210`, and 1000 ms for the `OPT3001DNPR`. The module is fully driven by compile-time feature flags; disabled sensors introduce no runtime overhead when excluded at configuration time.

Figure 3.3 illustrates the Sensor Module execution flow. It shows the periodic tick check and conditional execution of each sensor task based on its configured sampling period. The diagram highlights the non-blocking behavior, where each sensor updates the shared system data structure independently without delaying other tasks.

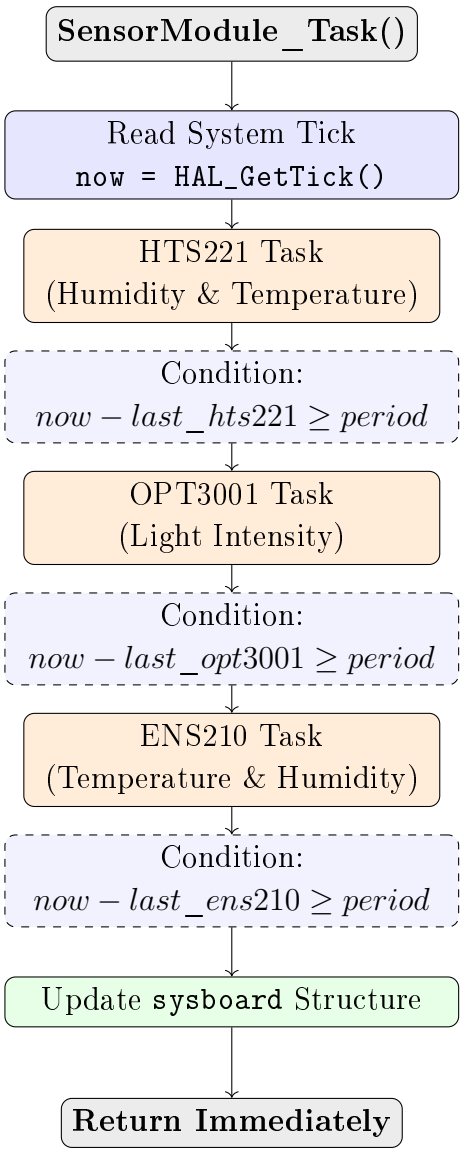

Figure 3.3: Sensor Module Periodic Execution Flow

Modbus Application Module

The Modbus module provides a simple interface between the Modbus communication layer and the system data. Upon receiving a Modbus master request, the module identifies what data is being accessed and links each request to its corresponding value that's stored in the system. Depending on the request type, it either reads data from the system or updates it, then returns the appropriate response. Overall, it acts as a routing layer that connects external Modbus messages to internal application variables in a structured and consistent way.

Figure 3.4 illustrates the Modbus module polling and data access flow. It shows how incoming Modbus requests are decoded and mapped to the corresponding object through a lookup in the object map. Depending on the object type, the request is dispatched to the appropriate read or write handler, which accesses the Sysboard global structure. The diagram highlights the dynamic register abstraction layer that decouples Modbus communication from hardware-specific data sources.

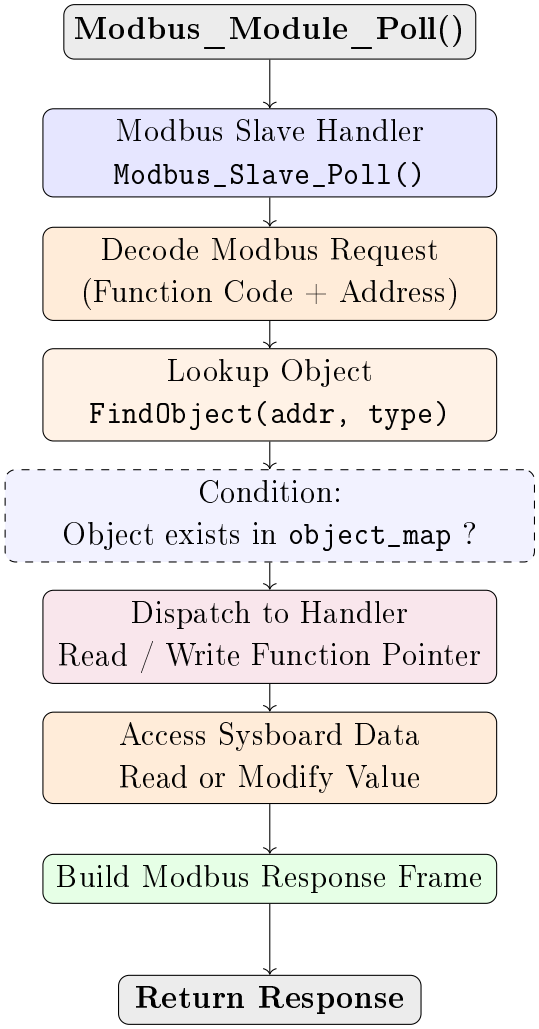

Figure 3.4: Modbus Module Polling and Object Access Flow

Frame Builder

The Frame Builder module is responsible for assembling the final output structure that the IOBOX platform transmits to external systems such as IoT platforms and UART devices. Rather than forwarding raw sensor readings directly, the platform encapsulates all acquired data within a structured binary frame whose layout is illustrated

in Figure 3.5. This design ensures that any receiver can clearly identify, validate, and decode the payload regardless of which peripheral is active in a given deployment.

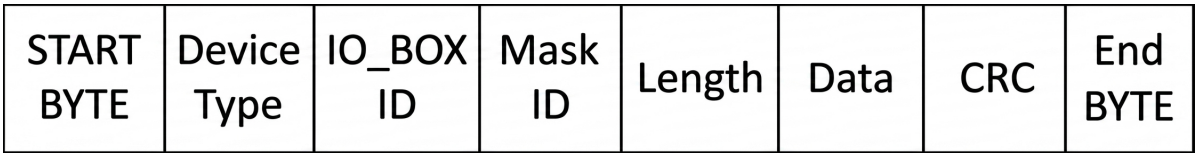

Figure 3.5: Structure of the IOBOX output frame

Figure 3.6 illustrates the frame construction workflow of this module. It shows how the periodic task retrieves the latest system snapshot from EEPROM and then distributes the data into parallel processing paths, including sensor data mapping and CRC computation. In parallel, protocol metadata such as frame delimiters and identifiers are prepared before being merged with the aggregated payload. The diagram highlights the final composition stage, where all components are assembled into a complete IoFrame and passed as a transmission-ready structure with an integrity-protected checksum.

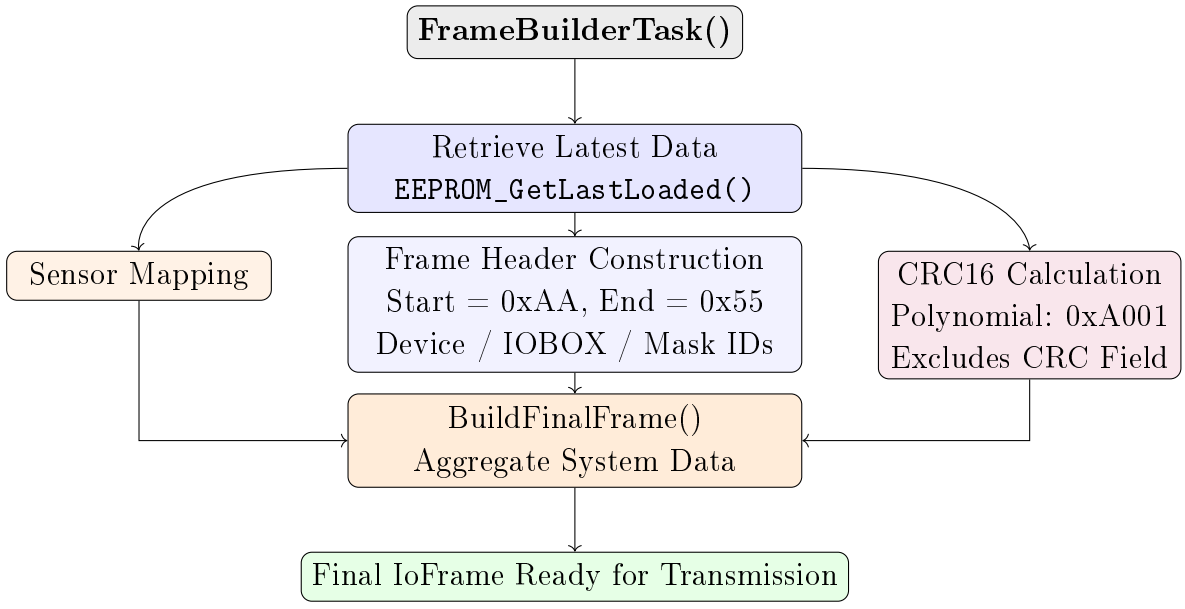

Figure 3.6: Frame Builder Module Workflow

The frame assembly routine fills each field with the most recently persisted system state, then computes and appends a CRC checksum over all preceding fields. The complete frame format is described in detail below.

Start Byte: A fixed synchronisation marker (`0xAA`) that signals the beginning of a valid frame to any receiver scanning an incoming byte stream.

Device Type: Identifies the category of the transmitting device, allowing the web server to host multiple types of embedded nodes simultaneously. In order to apply the proper frame decoding logic, the receiving gateway examines this field without knowing the sender's identity beforehand.

IOBOX ID: Carries the unique numeric identifier of the specific IOBOX unit. In multi-node deployments, this field allows the receiver to associate each frame with its physical origin without requiring any session or connection establishment.

Mask ID: A configuration bitmap indicating which sensors and communication modules are active in the current frame. Each bit maps to a specific module, as detailed in Table 3.3. The receiver uses this field to interpret the payload layout, ensuring flexible and forward-compatible parsing.

Table 3.3: Mask ID bit field definitions

Bit	Module	Logic 1 meaning
5	CAN sniffer	CAN data present in payload
4	HTS221TR Temperature	HTS221 temperature present
3	HTS221TR Humidity	HTS221 humidity present
2	ENS210 Temperature	ENS210 temperature present
1	ENS210 Humidity	ENS210 humidity present
0	OPT3001DNPR Illuminance	OPT3001 lux value present

Length: Provides the total size of the data field in bytes, making the frame self-describing. The receiver uses this field to locate the CRC and end byte without recalculating the payload size from the mask ID.

Data: Contains the measurement values and communication data collected during the most recent acquisition cycle where the Mask ID field's active bits completely control its contents.

CRC: A 16-bit CRC computed over all preceding fields using the CRC-16/IBM polynomial. The receiver validates it before processing the data; any mismatch indicates corruption and causes the frame to be discarded.

End Byte: A fixed end of message marker (`0x55`) providing an additional frame boundary check. Together with the Start Byte, it helps detect framing errors caused by a corrupted Length field, even when CRC validation alone wouldn't catch misalignment.

Table 3.4 summarises the complete custom frame field layout.

Table 3.4: IoFrame field summary

Field	**Value**	**Description**
Start Byte	`0xAA`	Frame synchronisation delimiter
Device Type	Compile-time constant	Node category identifier
IOBOX ID	Compile-time constant	Per-unit unique identifier
Mask ID	6-bit bitmap	Active module bitmap
Length	Variable	Data field length in bytes
Data	Variable	Active measurement payload
CRC	CRC-16/IBM	Integrity check over all preceding fields
End Byte	`0x55`	Frame termination delimiter

The frame assembly routine first retrieves the last persisted system state, then fills every field sequentially before calculating and inserting the CRC. This design ensures that the output frame always reflects the most recently stored platform state, providing continuity across power cycles and making the frame contents auditable through the non-volatile storage layer.

The software architecture described in this section provides a clean separation of concerns, compile-time configurability, and a deterministic execution model without the overhead of a real-time operating system. The following section presents the validation results obtained on the target hardware.

3.3 System Validation and Functional Testing

Each hardware interface was tested separately before any integration work began. The goal was simple: make sure every peripheral actually works before building anything on top of it. Tests were run directly on the STM32G474MBTx target board using STM32CubeIDE, with results printed through the SWO debug terminal to ensure a visual validation for evrey test .

3.3.1 ADC Validation

ADC channels were sampled in a single scan sequence to check that the conversion phase was set up correctly. The results were printed to the terminal as shown in Figure 3.7.

Channel 0 read zero, which was expected that pin was left unconnected. Every other channel returned a value somewhere in the 0–4095 range as expected .The scan ran without errors and the results looked physically reasonable, so the ADC driver was considered validated at that point.

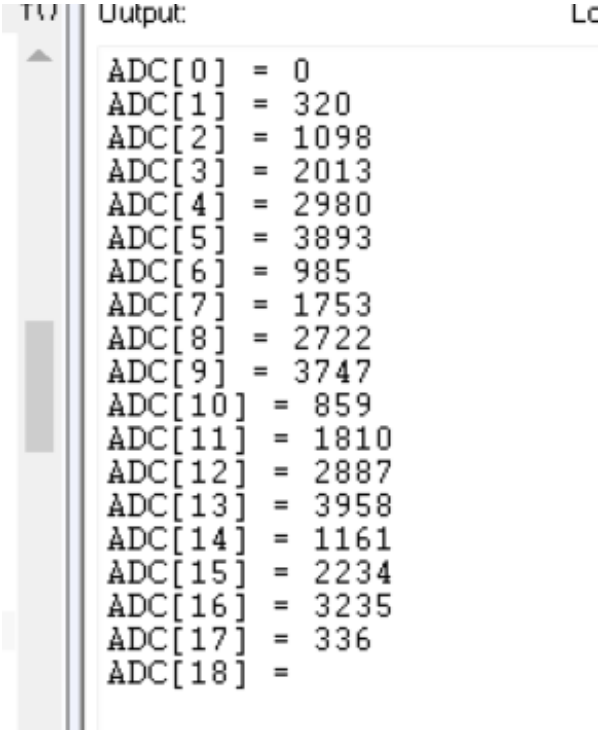

Figure 3.7: ADC raw output across 19 channels

3.3.2 SPI Communication Test

The SPI driver was tested by reading data from an esp32 SPI peripheral and printing each received byte to the IAR SWO debug terminal. Seven consecutive receive frames were captured each containing eight bytes as being sent from the esp32 (Figure 3.8). Clock polarity, phase, and bit order were all set and the received data matched the expected response format.That was enough to confirm the SPI bus was working.

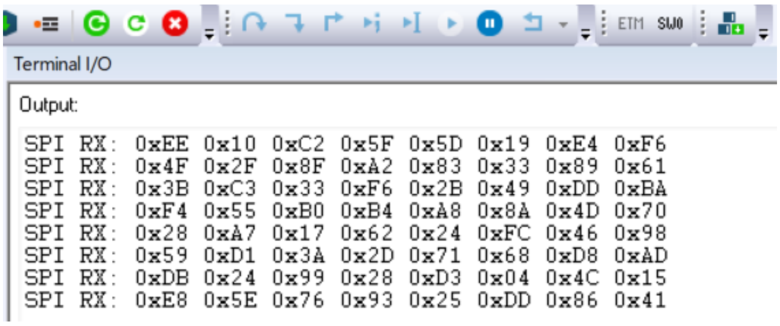

Figure 3.8: SPI receive frames captured on the debug terminal

3.3.3 CAN Standard Identifier Test

A CAN GPS module was connected to the board with a CAN transceiver and a 120 ohm resistor and configured to send frames with standard 11 bit identifiers at a fixed rate. The firmware was succeeded to print each received frame payload to the terminal.

Figure 3.9 shows the captured output. Three payload patterns cycled in a fixed sequence throughout the test as shown in the figure bellow so The part of the payload `DE AD BE EF` stays identical across all of them which confirms the payload was not getting corrupted on reception. The frame cycle matches exactly what was sent from the CAN GPS module side meaning our CAN module works perfectly.

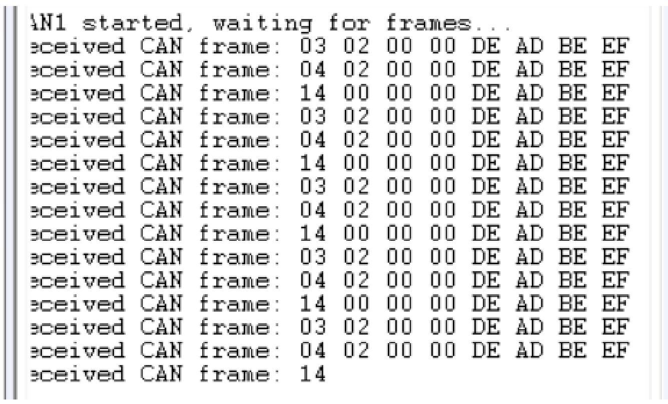

Figure 3.9: CAN frames received with standard 11-bit identifiers

3.3.4 CAN Extended Identifier Test

The same test was repeated using 29-bit extended identifiers to make sure the CAN filter was configured to accept that frame format as well.

in the test we configured the module to show evrey part of the frame not just the payload part and all received frames show the identifier `0x1752286` with `DLC=8`, as seen in Figure 3.10. The first two bytes of the payload increment across frames — `C6 14`, `C7 14`, `E5 14` which matches the counter field being incremented as we expect from the sender . The last four bytes stay fixed at `DE AD BE EF` as before confirming no corruption. The firmware printed the full 29-bit ID correctlyso the extended frame mode was working as expected.

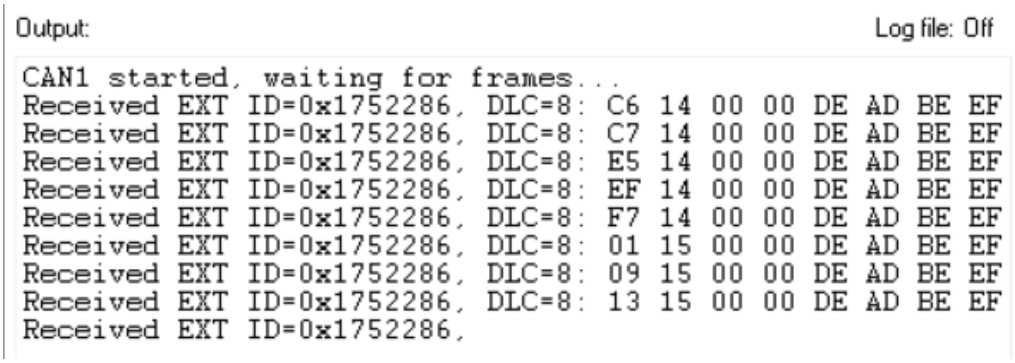

Figure 3.10: CAN frames received with extended 29-bit identifiers

3.3.5 ENS210 Humidity Validation

The ENS210 was tested by running the acquisition routine in a loop and watching the humidity output on the terminal. The sensor sits on I^2C multiplexer channel 2.

The first value printed was 0.00 %RH, which happened because the reading was taken before the sensor finished its first measurement cycle . the sensor was configured in single-shot mode. After that readings settled between 53.29 and 53.39 %RH which is a reasonable indoor humidity level for the conditions during testing. A wet paper was then held close to the sensor and the readings climbed to 83.59 %RH before dropping back down once it was removed. Figure 3.11 shows the full result. The sensor responded to the change the multiplexer channel selection worked and the conversion gave physically corrected numbers.

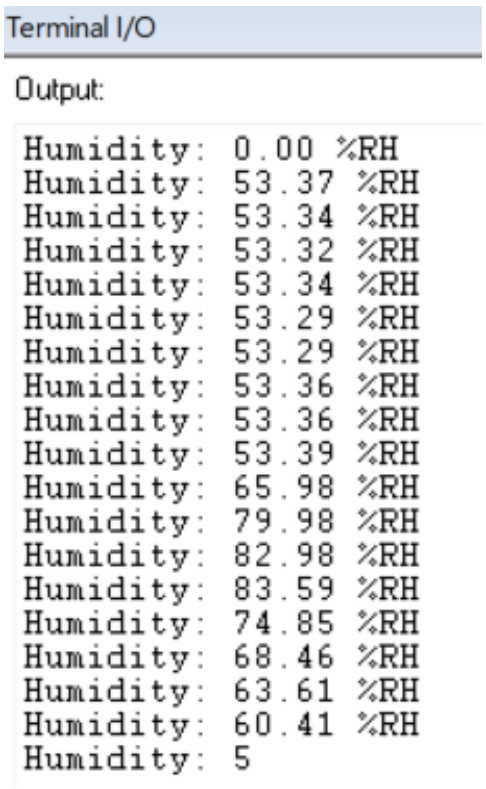

Figure 3.11: ENS210 humidity readings during ambient and induced humidity test

3.3.6 ENS210 Temperature Validation

The temperature output of the ENS210 was verified alongside the humidity test, using the same acquisition loop on multiplexer channel 2.

The first reading came back as -45.00 degreeCelsius, which is the value the conversion formula produces when the sensor has not yet completed its first measurement the raw register still holds its reset state at that point. From the second reading the output stabilised, cycling between 38.6 degreeCelsius and 39.96 degreeCelsius as shown in Figure 3.12. That range is slightly above typical room temperature because the sensor sits close to active components on the board and picks up some of the heat they dissipate. which confirms that the sensor is running correctly and that the temperature conversion is being applied successfully .

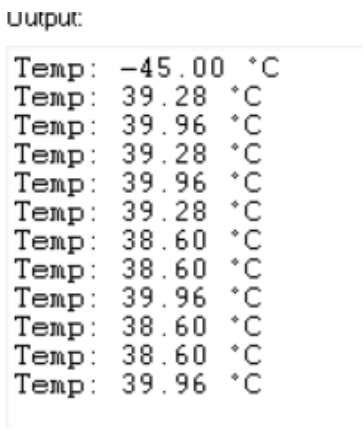

Figure 3.12: ENS210 temperature readings showing stabilisation after sensor warm-up

3.3.7 OPT3001 Sensor Validation

The OPT3001DNPR was tested by running acquisitions while changing the light level above the sensor and checking that the lux output responds to the changes correctly.

Figure 3.13 shows the raw register values alongside the computed lux results after conversion . Lux readings ranged from 63 to 493 during the test, which corresponds to the actual lighting level in the room lower values when a finger was placed over the sensor higher values when a lamp was brought closer. which validates our work with the adressing , the conversion and the good calibration .

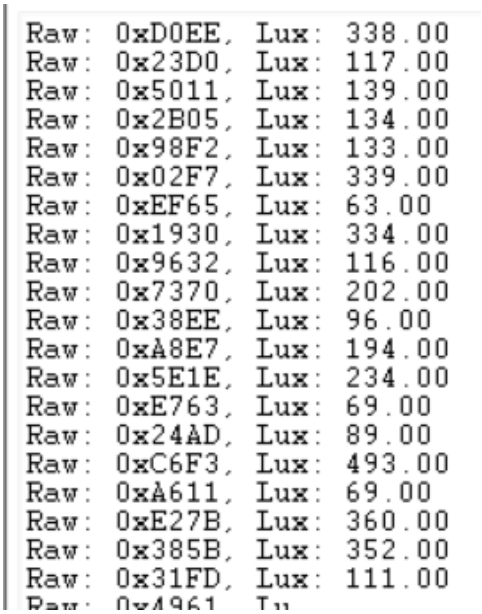

Figure 3.13: OPT3001 raw register values and computed lux output

3.3.8 HTS221TR Sensor Validation

The objective of this test was to confirm that the HTS221TR driver was correctly acquiring temperature and humidity data and exposing it through the communication interface.

Figure 3.14 shows the captured data with no missing response or errors appears across the entire captured sequence, confirming that the sensor data was being read and transmitted correctly.

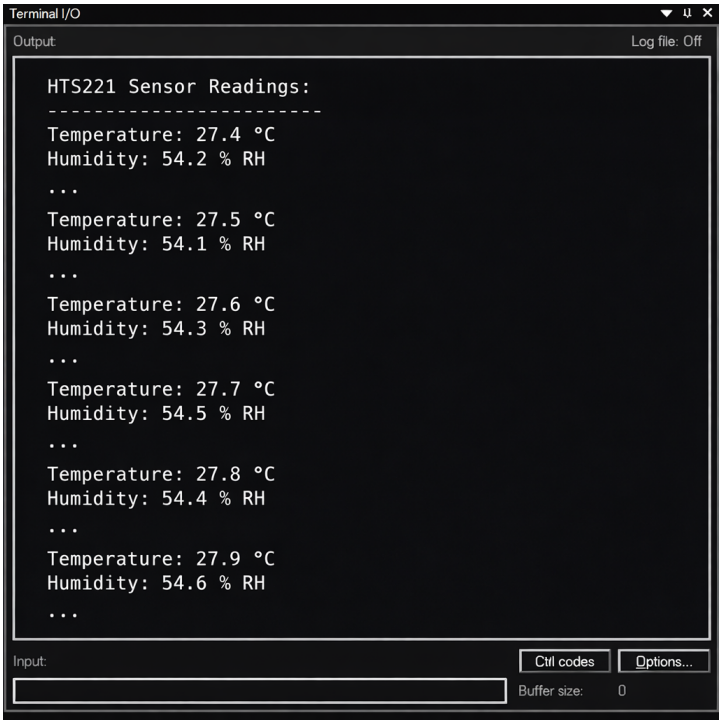

Figure 3.14: Communication traffic log during HTS221TR sensor validation

3.3.9 RS485 Communication Test

The RS485 communication was validated by connecting the STM32 test board to a PC via a serial terminal (COM6, 115200 bps) using a USB-to-RS485 adapter. The STM32 periodically transmitted sensor data over the RS485 bus, allowing real-time verification of correct data reception on the PC side.

Figure 3.15 shows the PC terminal output. After USART3 initialization and RS485 node startup, the system begins transmitting sensor data, which is successfully received and displayed, including temperature and humidity readings.

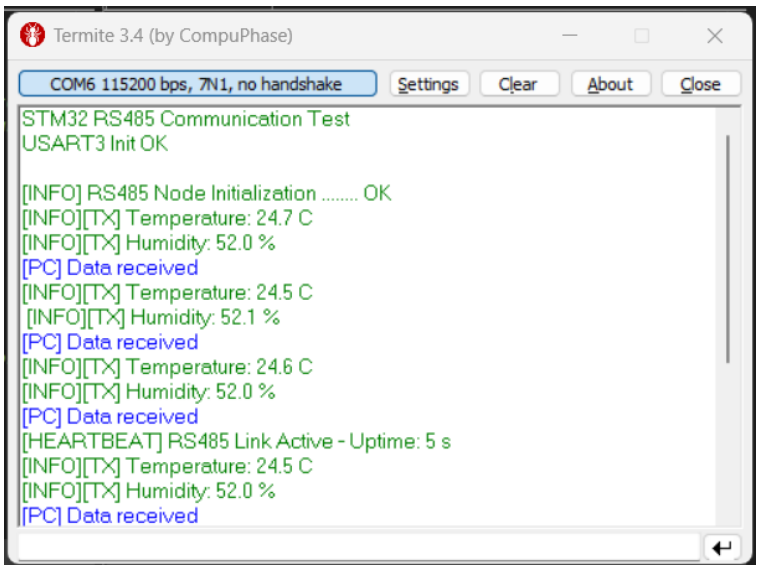

Figure 3.15: RS485 reception output on the PC terminal (Termite 3.4)

Validation was also performed directly on the STM32 side using the IAR EW Terminal IO interface. The logs confirm correct initialization and continuous transmission of sensor values, verifying proper RS485 operation in both transmit and receive modes.

Figure 3.16 shows the corresponding STM32 output, where initialization messages are followed by periodic sensor data logs confirming correct communication behavior.

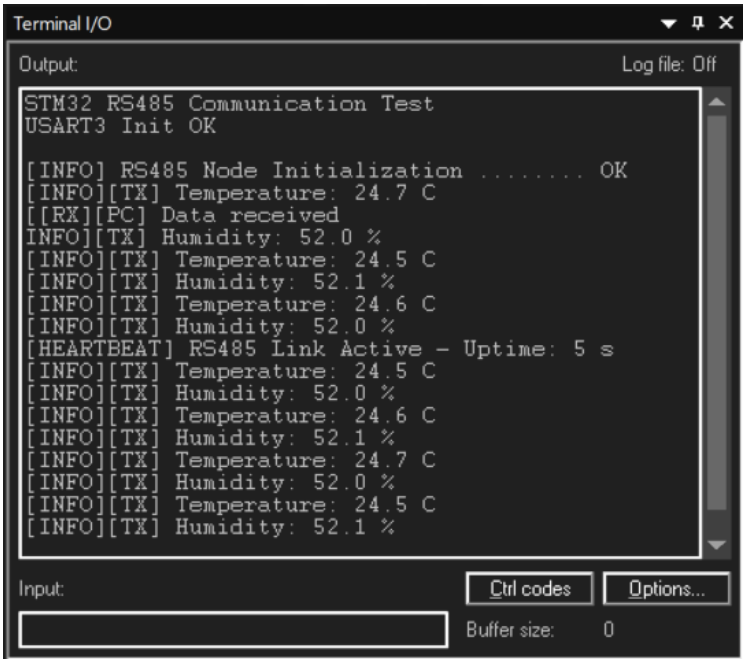

Figure 3.16: RS485 transmission log on the STM32 debug terminal

3.3.10 Modbus RTU Communication Test

The Modbus RTU slave implementation was validated using a PC-based Modbus master simulator (Modbus Poll) connected to the STM32 through an RS485 transceiver. The validation covered both read and write transactions in order to verify correct protocol decoding, data routing, and response generation. The results confirmed that the STM32 correctly interprets incoming Modbus RTU frames, updates the associated internal `Sysboard` data structure, and returns properly formatted responses compliant with the Modbus specification.

Figure 3.17 shows the main Modbus Poll interface used to establish communication with the STM32 slave, providing the overall test environment to initiate requests, monitor responses, and observe communication status continuously.

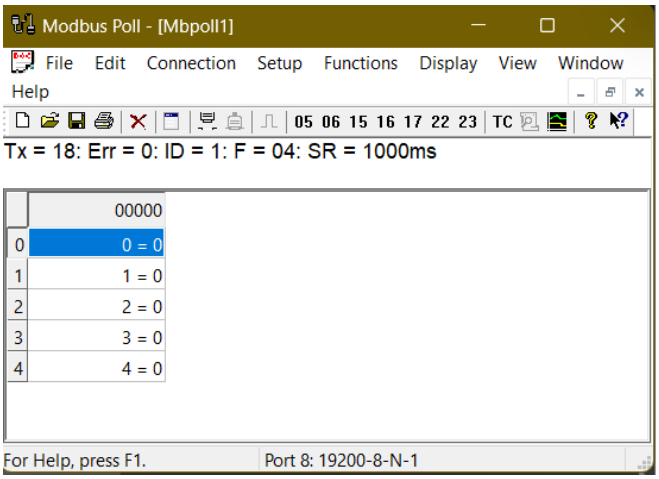

Figure 3.17: Modbus Poll master interface

Figure 3.18 shows the detailed configuration of the Modbus master request used during the test. The setup defines slave ID 1, starting register address 0, and a quantity of 5 registers using function code `FC04 (Read Input Registers)`, ensuring that multiple consecutive input registers are read in a single transaction.

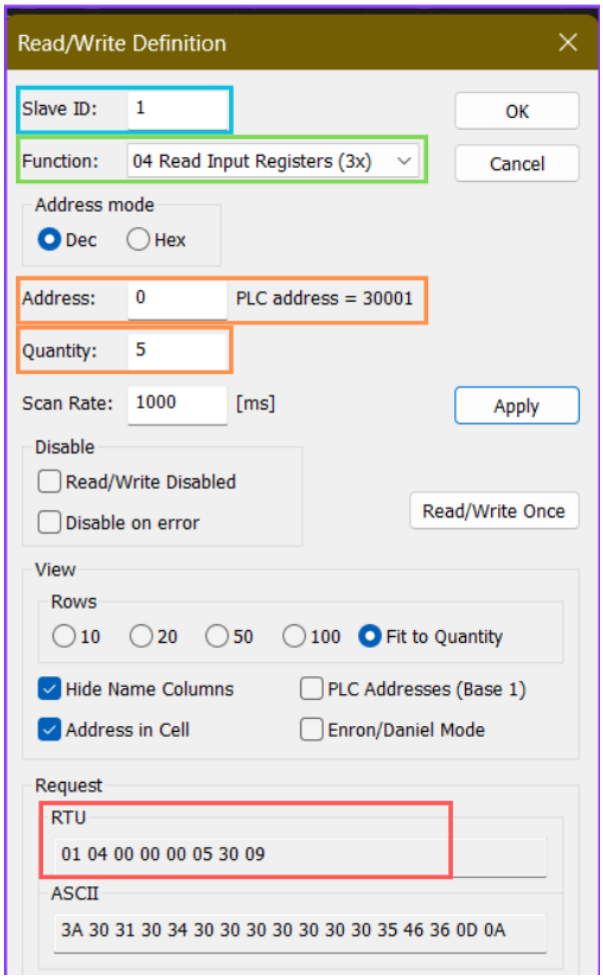

Figure 3.18: Configuration of FC04 Modbus request

Figure 3.19 illustrates the real-time RS485 communication trace captured during execution. The traffic log shows the request frame transmitted by the master, `01 04 00 00 00 05 30 09`, where `01` is the slave address, `04` corresponds to the `Read Input Registers` function code, `00 00` specifies the starting register address, and `00 05` requests five consecutive registers. The successful reception of a valid response confirms proper communication between the Modbus master and the STM32 slave over the RS485 link.

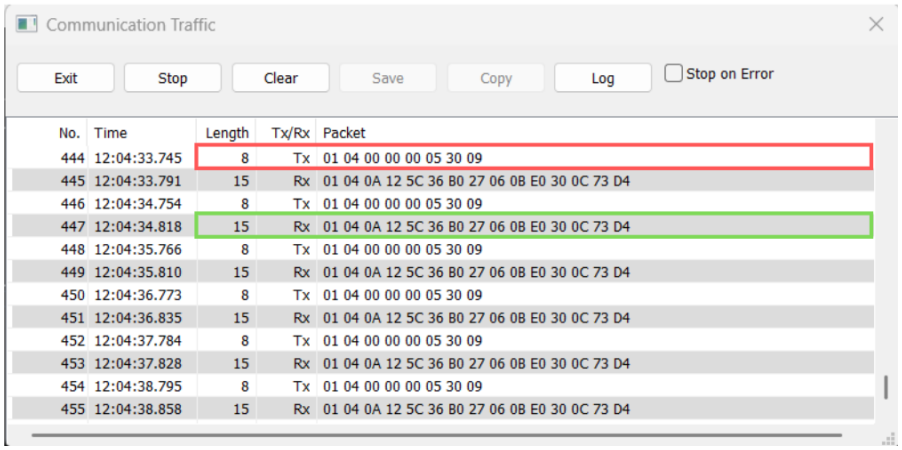

Figure 3.19: Modbus Poll communication trace

Figure 3.20 shows the STM32 debug terminal output during the transaction. The log confirms correct reception and parsing of the Modbus frame, execution of the slave handler, and access to the `Sysboard` global structure, which contains the application data exposed through the Modbus register map.

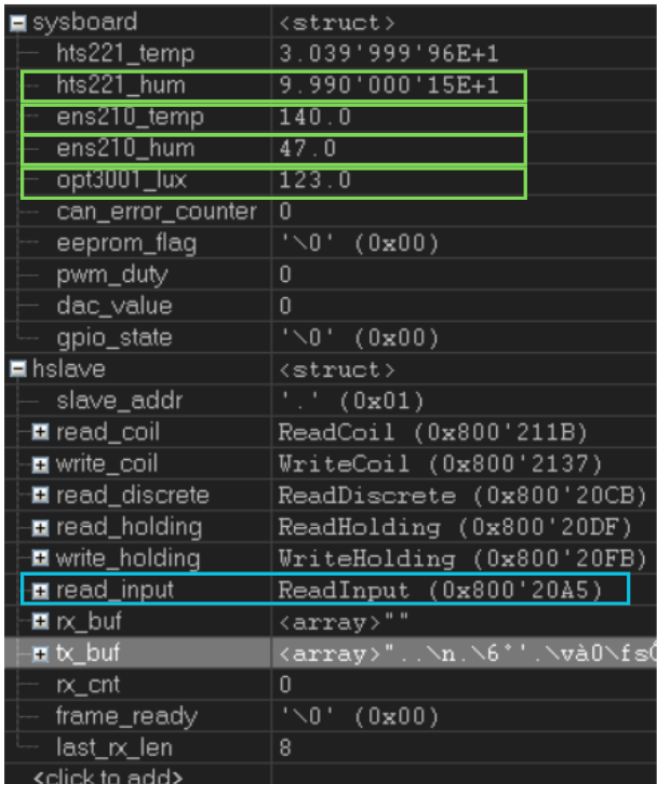

Figure 3.20: STM32 Modbus slave debug log

Figure 3.21 displays the raw transmit buffer generated by the STM32 before RS485 transmission. The response frame, `01 04 0A 12 5C 36 B0 27 06 0B E0 30 0C 73 D4`, contains the slave address (`01`), the echoed function code (`04`), and a byte count of `0A`, indicating ten data bytes corresponding to the five requested registers. The returned register values are `0x125C`, `0x36B0`, `0x2706`, `0x0BE0`, and `0x300C`, followed by the CRC field. The frame contents match the values stored in the `Sysboard` structure and confirm correct register mapping, data retrieval, and response generation by the Modbus slave implementation.

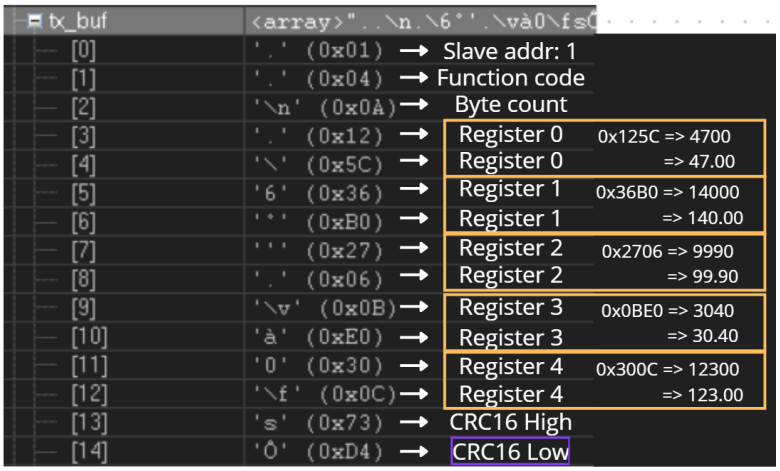

Figure 3.21: STM32 Modbus slave transmit buffer

3.4 Conclusion

This chapter presented the design and practical implementation of the IOBOX platform, covering its layered software architecture and the main firmware components from low-level hardware abstraction to high-level application logic. It detailed the implementation of the Manager and Application layers, including sensor acquisition, communication protocols, data processing, and frame construction, all driven by a modular and compile-time configurable design. The chapter concluded with practical validation on the STM32 target, confirming correct operation of all peripherals, communication interfaces, and overall system behavior under real test conditions.

GENERAL CONCLUSION AND PERSPECTIVES

This graduation project was carried out within Be Wireless Solutions as part of the attainment of the National Engineering Diploma in Electronics and Embedded Systems. work presented in this report addresses a concrete engineering challenge faced by Be Wireless Solutions: the absence of a common, reusable firmware base for industrial IoT deployments that previously required complete redevelopment for each new hardware configuration. The IOBOX platform was designed to eliminate that problem by providing a single, modular embedded system capable of adapting to heterogeneous sensor and communication environments without structural changes to the firmware. intended to serve as a shared and reusable base across future BWS deployments.

Throughout this report, we presented the different phases of our work. We began by establishing the general context of the project, introducing the host company, its activities, and the industrial need that motivated this work. We then defined the software architecture of the system, and explained the design choices that guided our implementation.

The development phase covered the construction of a layered firmware structure, from low-level peripheral drivers up to the application logic. We implemented sensor acquisition, industrial communication protocols, signal processing, and data persistence, all organized within a clean and extensible architecture. The platform was then subjected to a series of hardware validation tests that confirmed the correct behavior of every major component and interface on the target microcontroller.

This project required several months of sustained work, combining embedded software development, protocol integration, and systematic testing. It gave us the opportunity to face real engineering challenges from low-level driver debugging to architectural decisions and to develop a deliverable that holds genuine industrial value for the com-

pany.

Despite the results achieved in this first version, we believe that several improvements and extensions could strengthen the platform further, and we set them as perspectives:

- Migrate from the cooperative scheduler to a lightweight real time operating system to better handle time critical tasks.
- Extend the communication stack with support for additional industrial protocols such as Modbus TCP .
- Add remote firmware update capability allowing system parameters and feature flags to be adjusted at runtime rather than at compile time.
- Extend the platform with a secure communication layer, adding authentication and data encryption to meet industrial cybersecurity requirements.

BIBLIOGRAPHY

[1] ISIMM. consulted on 04/13/2026. http://www.isimm.rnu.tn/public/.

[2] BWS. consulted on 04/13/2026. https://bewireless-solutions.com/en/about-bws/.

[3] VS code. consulted on 04/13/2026. https://code.visualstudio.com/.

[4] IAR EW. consulted on 04/13/2026. https://www.iar.com/embedded-development-tools/iar-embedded-workbench.

[5] STM32CubeMX. consulted on 04/13/2026. https://www.st.com/en/development-tools/stm32cubemx.html.